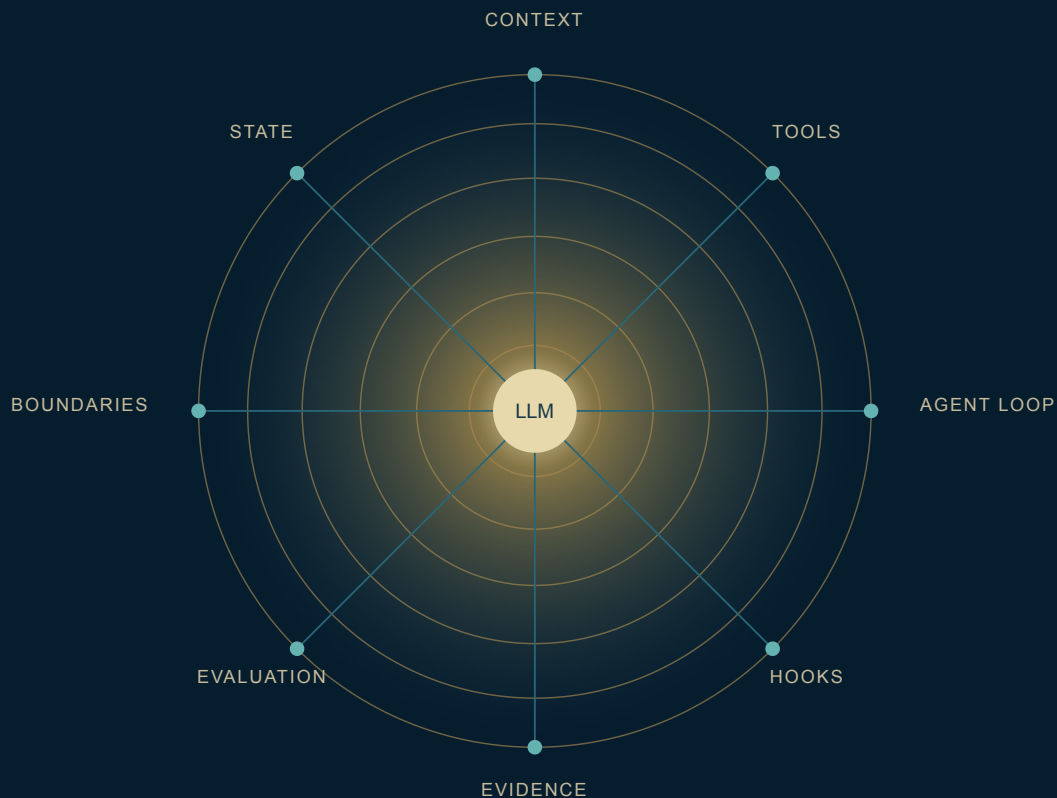


DAY 1 · SEPTEMBER 2026

AI HARNESS ENGINEERING

จาก LLM สู่ Agent ที่ลงมือทำ
มีขอบเขต มีหลักฐาน และตรวจสอบได้



AEKANUN THONGTAE

Lecture · 17 September 2026



DAY 1 · LECTURE-GROUNDED EDITION

AI Harness Engineering

เข้าใจโมเดล อ่านวงจร Agent
ออกแบบเครื่องมือและจุดควบคุมอย่างมีเหตุผล

Aekanun Thongtae

เรียบเรียงจากการสอนวันที่ 17 กันยายน 2026
ตรวจทานและค้นคว้าเพิ่มเติม: 18 กันยายน 2026
แหล่งหลัก: เอกสารทางการ OpenAI และ Anthropic

แนวทางของหนังสือ

หนังสือเล่มนี้พาผู้อ่านจากคำถามว่า “LLM ทำอะไรเองได้บ้าง” ไปสู่การอ่าน Agent Loop และการออกแบบจุดควบคุมรอบการเรียกเครื่องมือ เนื้อหาเรียบเรียงจากการสอนวันแรกให้เป็นบทอ่านต่อเนื่อง โดยรักษาตัวอย่างและเหตุผลของผู้สอน พร้อมปรับถ้อยคำที่อาจทำให้เข้าใจคลาดเคลื่อนตามเอกสารปฐมนิเทศ

เนื้อหาสามชั้นที่แยกจากกัน

ที่มา	ใช้ในเล่มอย่างไร
Lecture	แนวคิด อุปมา และกรณีศึกษาที่อธิบายในชั้นเรียนวันที่ 17 กันยายน
Official documentation	ใช้ยืนยันกลไก ขอบเขต และข้อยกเว้น โดยมีรหัสอ้างอิง [O] และ [A]
Engineering synthesis	ตัวอย่างวิเคราะห์และเกณฑ์ตรวจที่เรียบเรียงเพิ่มเพื่อเชื่อมแนวคิดกับการใช้งาน

วิธีอ่าน

อ่านบทที่ 1–3 เพื่อแยก Model, Harness และ Agent ก่อนเข้าสู่สัญญาของ Tool และ MCP ในบทที่ 4–5 จากนั้นอ่านเรื่อง Context และการควบคุมในบทที่ 6–7 ปิดท้ายด้วยการประเมินและการทบทวน Lab ในบทที่ 8 และภาคผนวก แต่ละบทจบด้วย Harness Design Check และคำถามที่ต้องอธิบายกลับได้

! ขอบเขตหลักฐาน

โค้ดในเล่มแยกเป็นรหัสเทียมประกอบคำอธิบาย และส่วนตัดจาก Source Code ที่ระบุแหล่งอ้างอิงชัดเจน การอ่านโค้ดไม่ใช้การทดสอบใหม่ หนังสือเล่มนี้ไม่ได้ทดลองรัน API หรือประเมินผลผู้เรียน

ตัดการสนทนาช่วงพัก ชื่อผู้เรียน ข้อมูลติดต่อ และรายละเอียดลูกค้าออก ภาพประกอบบางภาพนำมาจากหนังสือตัวอย่างของผู้เขียน ส่วนแผนภาพลำดับและตารางเรียบเรียงใหม่ให้ตรงกับเนื้อหาวันแรก

สารบัญ

ภาค 1 - จาก Model สู่ Agent

01	โมเดลยังไม่ใช่ Agent	5
02	LLM สร้างคำตอบอย่างไร	9
03	Agent Loop: จากคำตอบสู่การลงมือทำ	14

ภาค 2 - เครื่องมือและบริบท

04	Tool Calling: สัญญาระหว่าง โมเดลกับซอฟต์แวร์	19
05	MCP: ภาษากลางระหว่าง Agent กับเครื่องมือ	23
06	Context และความต่อเนื่องของงาน	27

ภาค 3 - การควบคุมและการตรวจสอบ

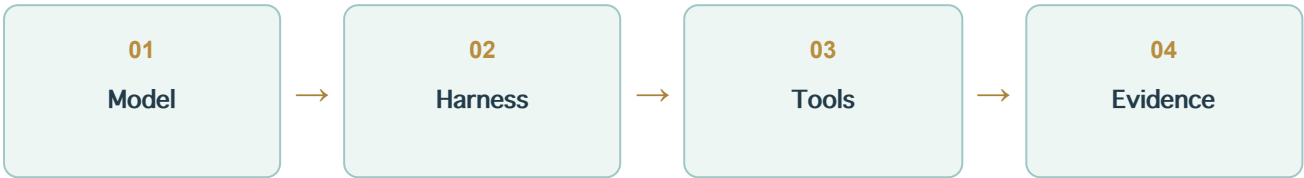
07	Hooks, Guardrails และขอบเขตการทำงาน	31
08	Evidence และการประเมิน Harness	36

บททวนและค้นต่อ

ภาคผนวก ก	อ่าน Lab ให้เห็นกลไก	40
ภาคผนวก ข	คำถามและแนวคำตอบ	41
ภาคผนวก ค	คำศัพท์ใช้งาน	42
เอกสารอ้างอิง	OpenAI	43

แผนที่ของวันแรก

หัวใจของวันแรกคือการมอง AI เป็นระบบหลายส่วน ความสามารถด้านภาษาของ โมเดลเป็นเพียงจุดเริ่มต้น ส่วนสิ่งที่เชื่อม โมเดลกับ โลกจริงต้องถูกออกแบบ ให้มีความหมายและตรวจสอบได้



คำสำคัญ	คำถามที่หนังสือตอบ	บท
Model	รับบริบทอะไร และสร้างผลลัพธ์แบบใด	1-2
Agent Loop	ใครเลือกก้าวถัดไป และหยุดเมื่อใด	3
Tools / MCP	อะไรถูกเรียกจริง และใช้สัญญาใดเชื่อมระบบ	4-5
Context / State	รอบถัดไปต้องรู้ข้อมูลอะไร	6
Hooks / Boundaries	กฎใดต้องตรวจ ก่อนผลกระทบเกิดขึ้น	7
Evidence / Evaluation	อะไรทำให้เราเชื่อว่าระบบทำงานถูก	8

จากการรันตามสู่การออกแบบเอง

เป้าหมายไม่ใช่จำคำศัพท์หรือสั่ง Coding Agent ว่า “สร้าง AI Agent ให้หน่อย” แล้วรับโค้ดทั้งหมด แต่คือ ระบุได้ว่าเครื่องมือควรรับค่าอะไร ผลใดถือว่าสำเร็จ ข้อมูลใดต้องเก็บกลับใน Context และการกระทำใดต้อง ถูกปฏิเสธหรือขออนุมัติ

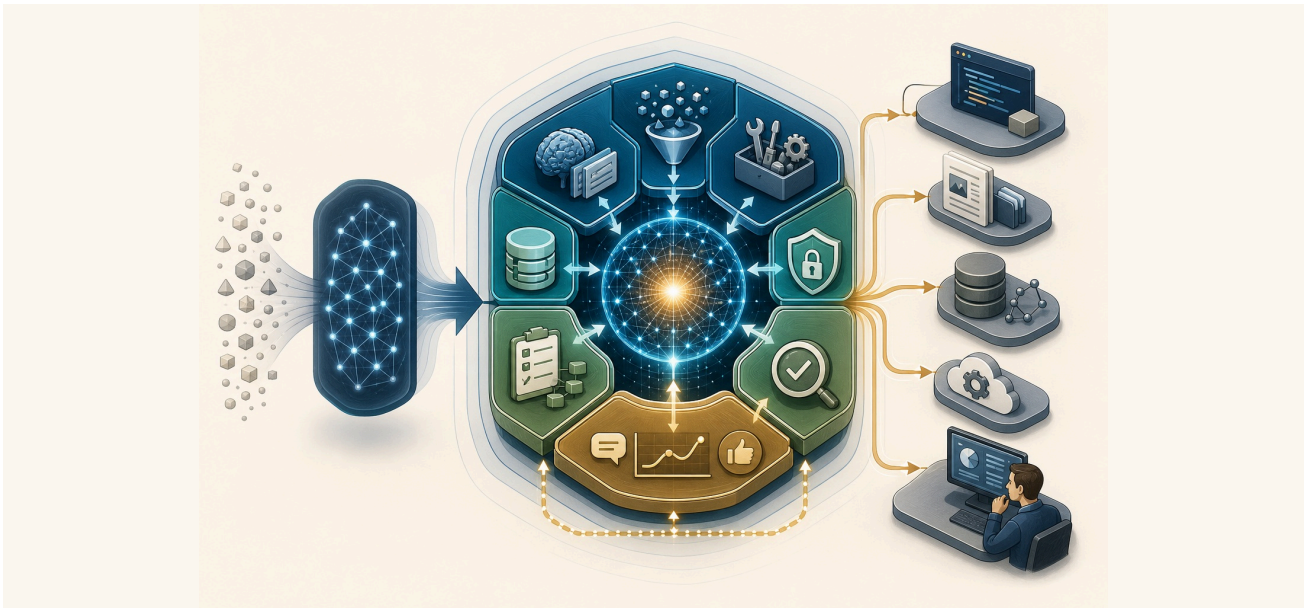
→ โจทย์ตลอดเล่ม

เมื่ออ่านจบ ให้เลือกงานหนึ่งงานแล้ววาดเส้นทางตั้งแต่คำขอของผู้ใช้ ถึง Tool Result และคำตอบสุดท้าย พร้อม วาดตำแหน่งที่ระบบต้องตรวจสอบสิทธิ์ ตรวจสอบข้อมูล และหยุด

การจัดลำดับในเล่มรวมหัวข้อที่ถูกกล่าวซ้ำระหว่างวันไว้ด้วยกัน เช่น Context และ Hook เพื่อให้อ่านต่อเนื่อง หัวข้อ Retrieval, Planning และ Multi-agent ที่กล่าวถึงเพียงประกอบจะไม่ถูกขยายเป็นหลักสูตรใหม่

บทที่ 1 - โมเดลยังไม่ใช่ Agent

แยกความสามารถของ โมเดลออกจากระบบที่พาโมเดลไปทำงาน



ภาพแนวคิดจากหนังสือตัวอย่างของผู้เขียน ใช้อธิบายความสัมพันธ์ ไม่ใช่ Trace ของระบบที่รันจริง

ข้อความที่บอกว่าทำแล้ว ยังไม่ใช่งานที่สำเร็จ

ผู้ช่วยอาจตอบว่า “ตรวจยอดขายให้แล้ว” ด้วยภาษาที่มั่นใจ แต่ถ้าไม่มีการเข้าถึงแหล่งข้อมูล เราไม่รู้ตัวเลขมาจากที่ใด การสอนวันแรกจึงเริ่มด้วยการแยก โมเดลออกจากแอปพลิเคชันและกลไกรอบโมเดล แทนที่จะเรียกทุกอย่างรวมกันว่า AI

LLM สร้างผลลัพธ์จากข้อมูลที่ถูกส่งเข้าไป ส่วนการอ่านฐานข้อมูล เปิดไฟล์ หรือเรียกบริการ ต้องมีซอฟต์แวร์ที่รับคำสั่งขอของโมเดลแล้วดำเนินการ การแบ่งหน้าที่นี้ทำให้เราหาจุดผิดพลาดได้ถูกชั้น [01]

◆ หลักคิด

ถามสองคำถามแยกกันเสมอ: โมเดลเสนออะไร และระบบได้ทำอะไรจริงแล้ว

Model, Harness และ Agent

มองสามคำนี้เป็นหน้าที่

Model คือส่วนที่ประมวลผลบริบทและสร้างผลลัพธ์ Harness คือซอฟต์แวร์ที่ห่อหุ้มโมเดลที่จัดบริบท เชื่อมเครื่องมือ เก็บสถานะ จัดการวงจร และบังคับข้อจำกัด ส่วน Agent ในความหมายที่ใช้ในเล่มคือระบบที่ให้โมเดลเลือกขั้นตอนและเครื่องมือภายในขอบเขตที่กำหนด

ส่วน	ความรับผิดชอบ	ตัวอย่างในวันแรก
Model	เสนอคำตอบหรือการเรียกเครื่องมือ	ขอเวลา และขอจำนวน 15×4
Harness	แปลคำร้องขอเป็นการทำงานที่อนุญาต	dispatch, messages, loop, hook
Environment	สิ่งที่ถูกอ่านหรือเปลี่ยนแปลง	เวลาในระบบ ฐานข้อมูล และไฟล์

สูตร “Agent = Model + Harness” มีประโยชน์ในการสื่อภาพรวม แต่ไม่ใช่สมการมาตรฐานที่ครอบคลุมทุกนิยาม ระบบยังต้องมีเป้าหมาย สภาพแวดล้อม และสิทธิ์ที่สัมพันธ์กับงานด้วย การมี Loop ในโค้ดอย่างเดียวไม่ได้พิสูจน์ว่าระบบมีอิสระตัดสินใจ

Workflow ต่างจาก Agent ตรงไหน

ถ้าโปรแกรมกำหนดทุกขั้นไว้ล่วงหน้า โมเดลเป็นหนึ่งในขั้นของ Workflow แต่ถ้าโมเดลเลือกที่จะเรียกเครื่องมือใดต่อจากผลที่เพิ่งได้รับ ระบบมีลักษณะ Agentic มากขึ้น Anthropic ใช้ความแตกต่างเรื่องผู้ควบคุมเส้นทางนี้แยก Workflow ออกจาก Agent [A1]

! แยกคำให้ชัด

อย่าใช้คำว่า “AI Chat ทุกตัวเป็น Agent” เป็นกฎตายตัว ต้องตรวจสอบความสามารถและการทำงานจริงของผลิตภัณฑ์นั้น

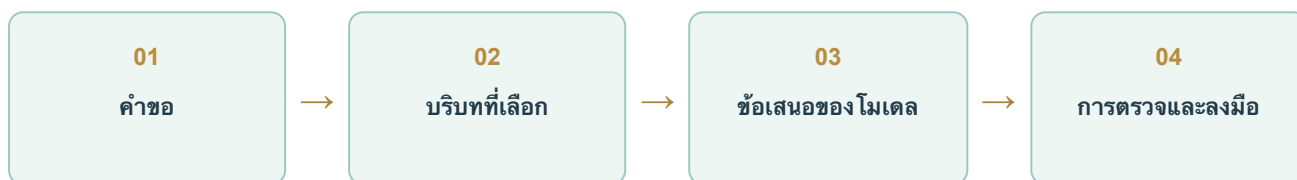
อ่านเพิ่มเติม [A1] Anthropic: Building effective agents · [O1] OpenAI: Function calling

Harness คือสิ่งที่เราปรับได้

เริ่มจากขอบเขตงานของนักพัฒนา

ในชั้นเรียน ผู้สอนชวนมองว่าทีมพัฒนาทั่วไปมักแก้ระบบรอบโมเดลได้ตรงกว่าแก่น้ำหนักภายในโมเดล ตัวอย่างคือจำกัดเครื่องมือ ปรับคำอธิบาย Tool เลือกข้อมูลที่ส่งเข้า Context และตรวจผลก่อนตอบผู้ใช้ สิ่งเหล่านี้เป็นการออกแบบพฤติกรรมของระบบ

หากระบบตอบผิดเพราะดึงข้อมูลผิด การเพิ่มประโยคย้ำใน Prompt อาจไม่แต่ละต้นเหตุ เราควรตรวจ Query, แหล่งข้อมูล และผลที่ส่งกลับก่อน หากระบบทำเกินสิทธิ์ กฎใน Runtime และสิทธิ์ของปลายทางมีความสำคัญกว่าคำสั่งว่า “ห้ามทำสิ่งอันตราย” เพียงอย่างเดียว



Harness ที่ดีไม่จำเป็นต้องใหญ่

เริ่มจากเส้นทางที่ง่ายพอแก้ไขได้ แล้วเพิ่มกลไกเมื่อมีเหตุผลรองรับ งานสรุปข้อความอาจไม่ต้องมีวงจรเรียก Tool หลายรอบ ส่วนงานที่ต้องค้นและตรวจหลายแหล่งอาจต้องมี Loop การเพิ่ม Memory หรือ Multi-agent โดยไม่มีปัญหาที่จะตอบ ทำให้มีจุดเสียและต้นทุนเพิ่ม [A1]

♦ จากห้องเรียนสู่การออกแบบ

ตัวอย่าง: คำขอให้คำนวณ 15×4 ไม่ต้องมีฐานข้อมูลความทรงจำข้ามเดือน แต่คำขอให้ตรวจยอดขายต้องมีแหล่งข้อมูลและนิยามช่วงเวลาที่ดี

ตัวชี้วัดที่สำคัญจึงไม่ใช่จำนวนองค์ประกอบในแผนภาพ แต่เป็นความสำเร็จของงาน ความสามารถในการจำกัดผลกระทบ และหลักฐานที่ใช้ตรวจย้อนหลัง

อ่านเพิ่มเติม [A1] Anthropic: Building effective agents · [O4] OpenAI: Guardrails and human review

จากความสามารถสู่ความรับผิดชอบ

คำถามเดียวกัน แต่หลักฐานคนละระดับ

ข้อความที่เห็น	ยังสรุปอะไรไม่ได้	หลักฐานที่ควรหา
“จะค้นฐานข้อมูล”	ยังไม่ทราบว่าจะเรียกจริงหรือไม่	Tool Call และสถานะ Execution
“ได้ผลลัพธ์แล้ว”	ยังไม่ทราบว่าครบหรือถูกขอบเขต	Result, ช่วงเวลา และแหล่งข้อมูล
“เสร็จแล้ว”	ยังไม่ทราบว่าตรงเป้าหมายหรือไม่	เงื่อนไขสำเร็จและผลตรวจ

ตัวอย่างเหล่านี้ทำให้เห็นว่าความมั่นใจของภาษาไม่เท่ากับความน่าเชื่อถือของระบบ การสร้าง Agent ต้องเชื่อมคำตอบเข้ากับเหตุการณ์ที่ตรวจได้ และแยกสิ่งที่ทำสำเร็จออกจากสิ่งที่ยังไม่ทราบ

✓ Harness Design Check

- **Harness รับผิดชอบ:** กำหนดเป้าหมาย จัดข้อมูล เชื่อมเครื่องมือ และบังคับขอบเขต
- **Model ตัดสินใจ:** เลือกการตอบหรือขั้นตอนถัดไปภายในความสามารถที่มี
- **Evidence ที่ต้องเก็บ:** คำขอ เครื่องมือที่เรียก ผลจริง และสถานะงาน
- **Failure ที่ต้องทดสอบ:** กล่าวว่าสำเร็จโดยไม่มีผลรองรับ หรือใช้ความสามารถเกินขอบเขต

→ ตรวจสอบความเข้าใจ

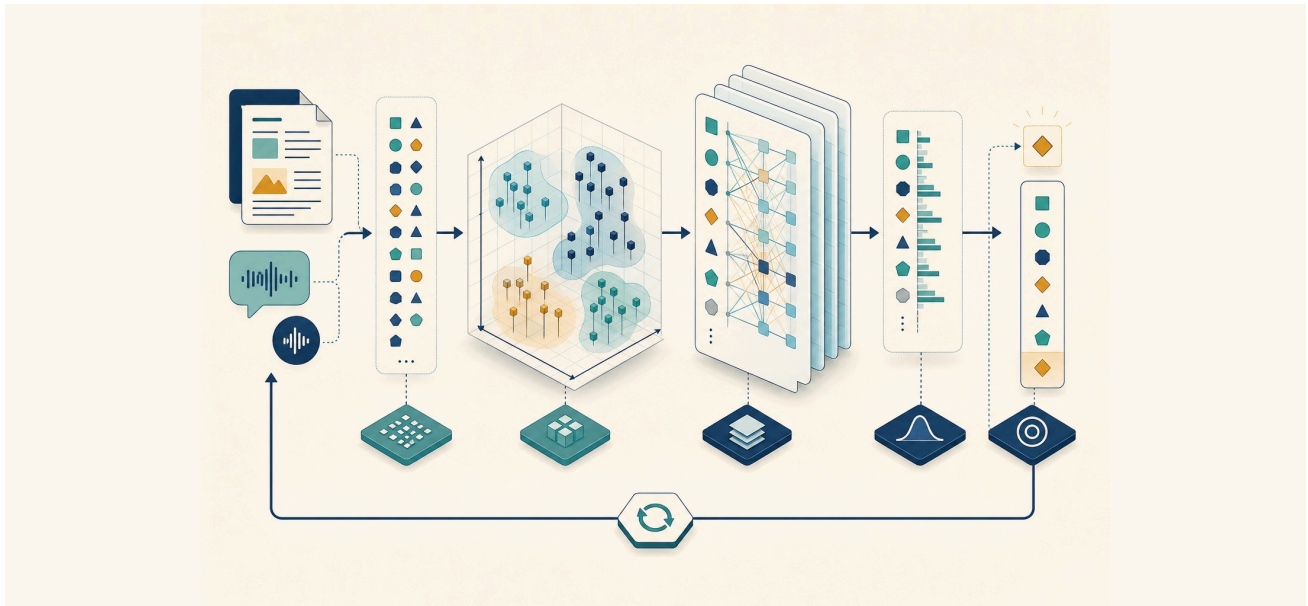
แอปหนึ่งมีช่องสนทนาและตอบได้ยาวมาก แต่ไม่มี Tool หรือการเข้าถึงข้อมูลภายนอก เราควรอธิบายความสามารถของแอปนั้นอย่างไร และมีหลักฐานอะไรที่ทำให้เปลี่ยนข้อสรุปได้?

บทถัดไปจะเปิดกล่องของการเรียก LLM ให้เห็นตั้งแต่ Prompt ไปถึงผลลัพธ์ เพื่อแยกหน้าที่ของโมเดล SDK และบริการที่อยู่ระหว่างทาง

อ่านเพิ่มเติม [O1] OpenAI: Function calling · [A1] Anthropic: Building effective agents

บทที่ 2 - LLM สร้างคำตอบอย่างไร

จาก Prompt และ Token ไปสู่ API ที่นักพัฒนาเรียกใช้



ภาพแนวคิดจากหนังสือตัวอย่างของผู้เขียน ใช้อธิบายความสัมพันธ์ ไม่ใช่ Trace ของระบบที่รันจริง

มอง LLM เป็นฟังก์ชันเพื่อเริ่มทำความเข้าใจ

ผู้สอนเสนอภาพง่าย ๆ ว่า LLM รับ Prompt แล้วคืนผลลัพธ์ วิธีมองนี้ช่วยให้นักพัฒนาตั้งคำถามได้ว่า Input คืออะไร ใครเป็นผู้ประกอบมัน และ Output ถูกนำไปใช้อย่างไร แต่ API จริงอาจรับรายการข้อความ ภาพ เสียง หรือโครงสร้างอื่น และคืนทั้งข้อความกับรายการ Tool Call ไม่ใช่ String เดียวเสมอไป [O1, O6]

ชื่อแอปที่อยู่บนหน้าจอ ชื่อโมเดลที่เลือก และชื่อบริการที่รับ API Request เป็นคนละส่วน การแก้ปัญหาต้องระบุให้ชัดว่ากำลังตรวจสอบส่วนใด

◆ หลักคิด

“ใช้ SDK ของใคร” ไม่เท่ากับ “กำลังเรียกโมเดลของใคร” ต้องดูปลายทางและความเข้ากันได้ของ API ด้วย

Prompt คือข้อมูลที่จัดให้ครบ

อย่าพูดคำว่า Prompt โดยไม่ระบุส่วน

ในบทเรียน ผู้สอนเน้นให้แยก User Prompt, System Prompt และข้อมูลประกอบ เพราะแต่ละส่วนเปลี่ยนด้วยเหตุผลต่างกัน คำขอของผู้ใช้บอกงานเฉพาะหน้า คำสั่งระดับระบบหรือแอปกำหนดพฤติกรรม ส่วนประวัติและ Tool Result ให้ข้อมูลที่จำเป็นต่อการทำงานต่อ

ส่วน	บทบาท	สิ่งที่ต้องระวัง
คำสั่งของระบบหรือแอป	กำหนดเป้าหมายและขอบเขต	ไม่ใช่สิทธิ์ที่ปลายทางบังคับจริง
ข้อความผู้ใช้	ระบุงานและเงื่อนไข	อาจมีข้อกำหนดที่ยังคลุมเครือ
ประวัติการสนทนา	รักษาความต่อเนื่อง	ข้อมูลเก่าอาจไม่ตรงปัจจุบัน
คำอธิบาย Tool และผล Tool	บอกความสามารถและหลักฐาน	ไม่ควรยกระดับข้อความภายนอกเป็นคำสั่ง

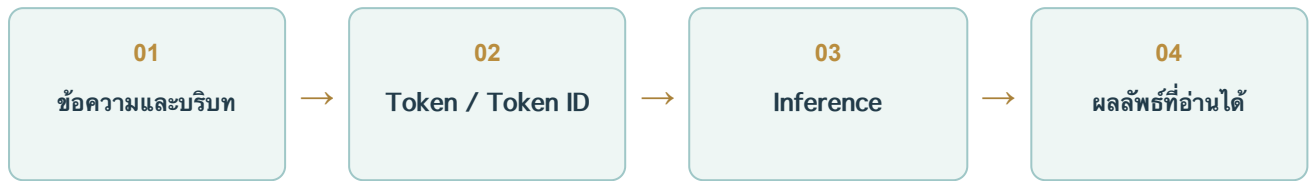
ชื่อ Role และการวาง Instruction แตกต่างตาม API หนังสือใช้คำเรียกเพื่ออธิบายหน้าที่ ไม่ใช่กำหนดว่า Request ทุกค่ายต้องมี Field ชุดเดียวกัน ก่อนต่อบริการจริงต้องอ่าน Schema ของปลายทางนั้น

Context กว้างกว่าคำถามล่าสุด

ถ้าผู้ใช้ถามว่า “แล้วปีที่แล้วละ” ระบบต้องรู้ว่าก่อนหน้านั้นคุยเรื่องอะไร หากส่งเฉพาะข้อความล่าสุด โมเดลอาจไม่มีข้อมูลพอให้ตีความ การรักษาประวัติหรืออ้างอิงถึงสถานะการสนทนาเป็นงานของชั้นแอปและบริการ [02]

อ่านเพิ่มเติม [02] OpenAI: Conversation state · [A2] Anthropic: Effective context engineering for AI agents

Tokenization, Inference และ Decoding



แบ่งข้อความ ไม่ใช่สร้างความหมายทั้งหมดในขั้นเดียว

Tokenization แปลงข้อความเป็นหน่วยย่อยและรหัสที่โมเดลใช้ ซึ่ง Token ไม่จำเป็นต้องเท่ากับหนึ่งคำ โดยเฉพาะภาษาไทย จึงไม่ควรนำจำนวนคำไปคิดเป็นจำนวน Token โดยตรง ส่วน Embedding เป็นการแทนข้อมูลด้วยเวกเตอร์ตัวเลข เป็นคนละแนวคิดกับการแบ่งข้อความเป็น Token

ในภาพอธิบายของวันแรก Inference คือช่วงที่โมเดลประมวลผล Input แล้วสร้าง Output Token ส่วนการแปลง Token ID กลับเป็นข้อความทำให้แอปแสดงคำตอบที่มนุษย์อ่านได้ กระบวนการภายในจริงมีรายละเอียดมากกว่าแผนภาพ แต่การแยกขั้นเช่นนี้พอสำหรับการอ่านเส้นทางข้อมูลของ Lab

คำตอบนั้นแปรได้ ต้องวัดระดับระบบ

การถามเหมือนเดิมอาจได้ผลต่างกัน อย่างอธิบายว่าเกิดเพราะ “เดาคำถัดไป” เพียงเหตุเดียว เพราะวิธีเลือก Token การตั้งค่า รุ่น โมเดล บริบท และระบบประมวลผลล้วนเกี่ยวข้อง หนังสือจึงใช้เป้าหมายว่าทำให้ระบบเชื่อถือได้ขึ้น แทนการรับประกันคำตอบเหมือนเดิมทุกตัวอักษร [03]

! ความสม่ำเสมอ ≠ ความถูกต้อง

การลดความสับสนไม่ได้รับประกันว่าคำตอบถูก และคำตอบที่เหมือนเดิมทุกครั้งก็อาจผิดเหมือนเดิมทุกครั้งได้

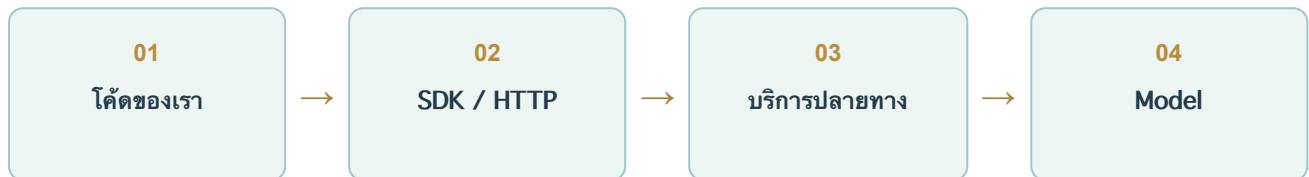
ใน Lab ให้บันทึก Input จริง รุ่น โมเดล ผลลัพธ์ และการตั้งค่าไว้ก่อนเปรียบเทียบ มิฉะนั้นเราอาจกำลังเทียบคนละเงื่อนไขโดยไม่รู้ตัว

อ่านเพิ่มเติม [03] OpenAI: Evaluation best practices · [06] OpenAI: Using tools

Model, Provider, Broker และ SDK

อ่านเส้นทางให้ถูกก่อน โทษโมเดล

วันแรกแบ่งช่องทางเข้าถึงโมเดลเป็นการรันด้วยตนเอง การเรียก Provider โดยตรง และการผ่าน Broker แนวคิดนี้ช่วยชี้ว่าคำขอผ่านชั้นใดบ้าง บริการกลางอาจช่วยทำรูปแบบ Request และ Response ให้ใกล้เคียงกัน แต่ไม่ควรถือว่าความสามารถทุกโมเดลเหมือนกัน



คำ	หน้าที่	สิ่งที่ไม่ควรเหมารวม
Model	ประมวลผล Input และสร้าง Output	ไม่ใช่หน้าจอสอนทนาทั้งแอป
Provider	ให้บริการรันโมเดล	อาจไม่ใช่ผู้พัฒนาโมเดลเสมอไป
Broker / Gateway	เป็นตัวกลางในการเข้าถึงบริการ	ไม่รับประกัน Feature Parity
SDK	ช่วยสร้าง Request และอ่าน Response	ไม่ใช่ Inference Engine เอง

การลงท้าย URL ด้วย /v1 ไม่พอพิสูจน์ว่า API เข้ากันได้ ต้องตรวจ Endpoint, Field, Role, Tool Schema, Streaming และ Error Format การใช้ OpenAI SDK กับปลายทางอื่นได้จึงขึ้นกับความเข้ากันได้ของปลายทาง ไม่ใช่ชื่อ SDK เพียงอย่างเดียว

♦ บทเรียนจากการอ่าน Lab

หาก API เปลี่ยนแล้วโปรแกรมอ่านคำตอบไม่ได้ ให้เก็บ Error และ Payload ที่ตัดข้อมูลล้นออก ตรวจสอบ Request/Response ก่อนสรุปว่าโมเดล “ไม่เก่ง”

เล่มนี้ไม่ใช่ราคาหรืออันดับโมเดลจากการสาธิตเป็นข้อเท็จจริงปัจจุบัน การเลือกบริการต้องตรวจความสามารถที่โจทย์ต้องใช้และวัดภายใต้เงื่อนไขเดียวกัน

อ่านเพิ่มเติม [O1] OpenAI: Function calling · [O6] OpenAI: Using tools

สิ่งที่ Lab เรียก LLM ต้องทำให้เห็น

รันได้เป็นจุดเริ่มต้น ไม่ใช่ข้อสรุปทั้งหมด

Lab ช่วงแรกแสดงคำถาม คำตอบ Token Usage เวลา และค่าใช้จ่าย จุดเรียนรู้คือแยก Input, Output และ ต้นทุนออกจากกัน พร้อมตรวจว่าการประมวลผลเกิดที่บริการใด ตัวเลขค่าใช้จ่ายต้องรู้ว่ามาจาก Field ของ บริการหรือการประมาณด้วยสูตร ไม่ใช่มีหน่วยดอลลาร์แล้วจะถือว่าเป็นยอดเรียกเก็บจริงทันที

ระหว่างสอนมีผลที่ต่างกันระหว่างเครื่องเรื่องการแสดงค่าใช้จ่าย ก่อนแก้ด้วยโค้ดเวอร์ชันที่ตรงกัน บทเรียนเชิง วิศวกรรมคือเก็บ Branch/Commit, Directory, Dependency และ Environment ให้ชัด การเปรียบเทียบ ที่ใช้คนละเวอร์ชันอาจสะท้อนความต่างของ โค้ดมากกว่าความต่างของ โมเดล

✓ Harness Design Check

- **Harness** รับผิดชอบ: ประกอบ Request จัดการสถานะ ตรวจ Error และรายงานต้นทุนตามที่มีหลักฐาน
- **Model** ตัดสินใจ: สร้างคำตอบหรือรายการ Output ที่ API รองรับ
- **Evidence** ที่ต้องเก็บ: เวอร์ชันโค้ด ปลายทาง รันโมเดล Request, Response, Usage และเวลา
- **Failure** ที่ต้องทดสอบ: ตั้งค่าไม่ครบ Schema ไม่ตรง Rate Limit และอ่านผลผิด Field

→ ตรวจสอบความเข้าใจ

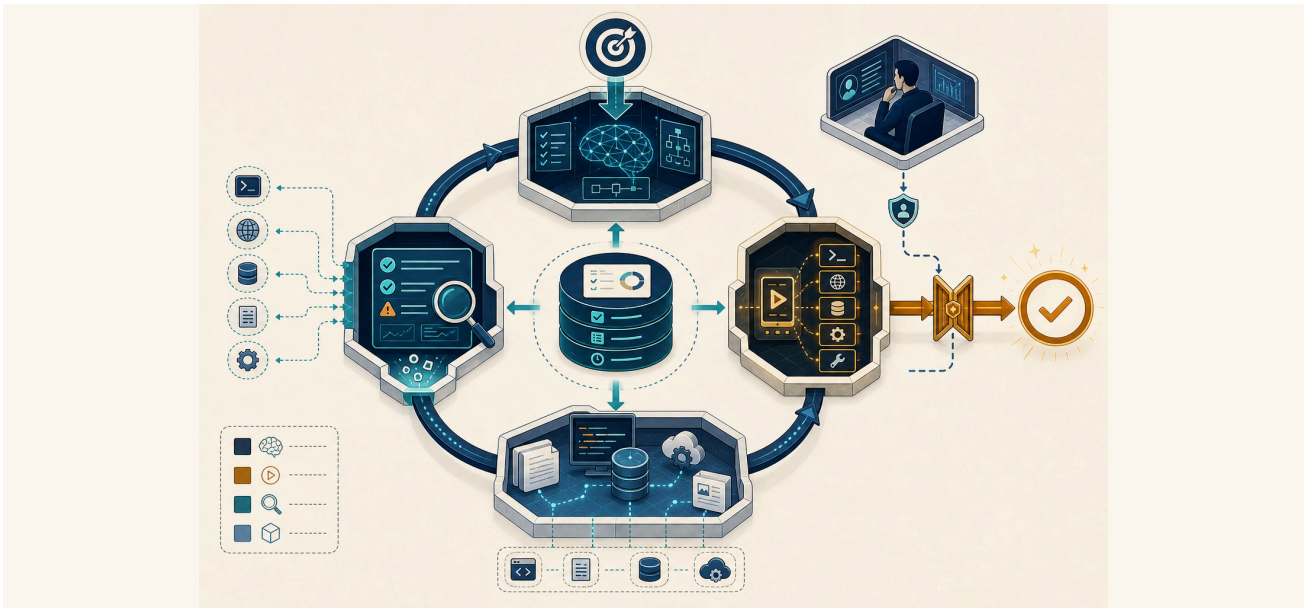
ถ้า SDK คืน Response Object ที่มีข้อความและ Tool Call พร้อมกัน โค้ดที่อ่านเฉพาะข้อความอาจพลาดอะไร? แล้วจะตรวจด้วยหลักฐานใด?

เมื่อแยกการเรียก LLM หนึ่งครั้งได้แล้ว ขั้นตอนต่อไปคือใช้ผลนั้นขับเคลื่อนวงจรการทำงานหลายรอบ ซึ่งเป็นหัวใจของ Agent Loop

อ่านเพิ่มเติม [01] OpenAI: Function calling · [02] OpenAI: Conversation state

บทที่ 3 - Agent Loop: จากคำตอบสู่การลงมือทำ

อ่านวงจรตัดสินใจ เรียกเครื่องมือ รับผล และตรวจเป้าหมาย



ภาพแนวคิดจากหนังสือตัวอย่างของผู้เขียน ใช้อธิบายความสัมพันธ์ ไม่ใช่ Trace ของระบบที่รันจริง

ผลของรอบนี้เป็นข้อมูลของรอบถัดไป

ตัวอย่างถามเวลาปัจจุบันและคำนวณ 15×4 ทำให้เห็นการทำงานที่เกินกว่าคำตอบครั้งเดียว โมเดลอาจขอ Tool สองตัว Harness เรียกฟังก์ชัน แล้วส่งผลกลับให้โมเดลประกอบคำตอบ การมีผลจริงกลับเข้ามานี้ทำให้การตอบ เชื่อมกับสภาพแวดล้อม [01]

คำว่า Think-Act-Observe ในชั้นเรียนเป็นกรอบช่วยอ่านวงจร ไม่ได้หมายความว่าต้องเปิดเผยเหตุผลภายใน โมเดลทั้งหมด สิ่งที่เราตรวจได้คือข้อเสนอ Tool, Arguments, ผลการทำงาน และสิ่งที่เกิดในรอบถัดไป

◆ หลักคิด

Loop ต้องมีทั้งทางเดินต่อ ทางจบ และทางหยุดเมื่อมีปัญหา

แกะหนึ่งรอบของ Agent

แยกการขอให้ทำออกจากการทำงานจริง

จังหวะ	สิ่งที่เกิด	หลักฐานที่สังเกตได้
ตัดสินใจ	โมเดลสร้างคำตอบหรือ Tool Call	รายการ Output และ Arguments
ตรวจ	Harness ตรวจชื่อ ค่า และสิทธิ์	ผลการตรวจและกฎที่ใช้
ลงมือ	Runtime เรียกฟังก์ชันจริง	สถานะสำเร็จ/ล้มเหลวของ Tool
รับผล	บันทึกผลและผูกกับคำร้องขอ	Call ID และ Tool Result
ประเมินต่อ	โมเดลหรือระบบเลือกขั้นถัดไป	คำตอบ Tool Call ใหม่ หรือสถานะหยุด

การเรียกโมเดลหนึ่งครั้งอาจได้หลาย Tool Call และไม่ได้แปลว่าต้องรันพร้อมกันเสมอ หากการเรียกที่สองต้องใช้ผลของครั้งแรก ต้องรักษาลำดับตามความสัมพันธ์ของข้อมูล การทำ Parallel ควรใช้เฉพาะงานที่แยกจากกันได้จริง

Observation ไม่ใช่การทบทวนที่เกิดขึ้นเอง

ถ้าระบบไม่ส่ง Tool Result กลับ โมเดลไม่มีหลักฐานใหม่นั้นสำหรับรอบต่อไป การ append ผลกลับเข้า messages ใน Lab จึงไม่ใช่รายละเอียดประกอบ แต่เป็นกลไกสำคัญที่ทำให้รอบถัดไปสัมพันธ์กับสิ่งที่ทำไปแล้ว

! ตรวจสอบผลตามเป้าหมาย

Tool ส่งสถานะสำเร็จทางเทคนิค ไม่ได้หมายความว่างานของผู้ใช้สำเร็จ เช่น Query รันได้แต่เลือกผิดปี ระบบต้องตรวจสอบความหมายของผลด้วย

อ่านเพิ่มเติม [O1] OpenAI: Function calling · [O2] OpenAI: Conversation state

อ่าน Loop ให้เห็นทางออกทุกทาง

รหัสเทียมต่อไปนี้แสดงความรับผิดชอบ ไม่ใช่โค้ดพร้อมรัน และไม่ผูกกับ API หรือชื่อฟังก์ชันของ Lab เวอร์ชันใดโดยเฉพาะ

```
state = initial_request_and_constraints
repeat within step, time, and cost budgets:
    output = ask_model(context_from(state), allowed_tools)
    record(output)
    if output requests tools:
        for call in output.tool_calls:
            validate_schema_and_authorization(call)
            result = execute_or_report_failure(call)
            state = append_call_and_result(state, call, result)
        continue
    if completion_is_supported(output, state):
        return final_answer
    stop_with_incomplete_status()
stop_with_budget_status()
```

ไม่เรียก Tool ต่อ ไม่เท่ากับสำเร็จโดยอัตโนมัติ

การไม่มี Tool Call อาจเป็นคำตอบสุดท้าย การปฏิเสธ หรือคำตอบที่ยังไม่พอ ต้องใช้เงื่อนไขสำเร็จของงานช่วยตัดสิน ในทางกลับกัน การครบ Max Steps หมายถึงถึงขีดจำกัด ไม่ควรเปลี่ยนสถานะเป็นสำเร็จเพื่อให้โปรแกรมจบสวย

งบจำนวนรอบ เวลา และค่าใช้จ่ายช่วยจำกัดงานที่ไม่ก้าวหน้า หากเกิด Error ชั่ว การวนต่อแบบเดิมอาจเพียงเพิ่มต้นทุน ระบบควรบอกว่าอะไรล้มเหลว อะไรยังไม่ได้ทำ และข้อมูลใดจำเป็นต่อการดำเนินการต่อ

◆ หลักคิด

การหยุดพร้อมหลักฐานว่า “ยังทำไม่สำเร็จ” น่าเชื่อถือกว่าการสร้างคำตอบให้ดูเหมือนเสร็จ

อ่านเพิ่มเติม [O1] OpenAI: Function calling · [O4] OpenAI: Guardrails and human review

วงจรที่ดีต้องตรวจย้อนหลังได้

จาก Tool Result สู่คำตอบสุดท้าย

ในตัวอย่างเวลาและเครื่องคิดเลข ให้ตามรอยว่าโมเดลเลือกชื่อ Tool ถูกหรือไม่ Arguments เป็นสิ่งที่ Tool รับผิดชอบหรือไม่ และคำตอบสุดท้ายใช้เวลาที่ได้กลับกับผลคำนวณ 60 จริงหรือไม่ ไม่ควรใช้ข้อความว่า “ผมเรียก Tool แล้ว” เป็นหลักฐานแทน Trace

หาก Tool คืน Error การตอบควรสะท้อนข้อจำกัดนั้น เช่น ระบุว่าไม่สามารถตรวจเวลาจริงได้ในรอบนี้ แทนการเติมเวลาที่คาดเดาเอง ความถูกต้องจึงครอบคลุมทั้งกรณีสำเร็จและกรณีที่ระบบต้องยอมรับว่าไม่มีหลักฐาน

✓ Harness Design Check

- **Harness รับผิดชอบ:** ชับ Loop ตรวจงบประมาณ บันทึก Call/Result และแยกสถานะจบ
- **Model ตัดสินใจ:** เลือกว่าจะตอบ เรียก Tool เพิ่ม หรือร้องขอข้อมูลที่ขาด
- **Evidence ที่ต้องเก็บ:** ลำดับรอบ Call ID ผล Tool เหตุผลการหยุด และผลตรวจความครบ
- **Failure ที่ต้องทดสอบ:** วงซ้ำไม่ก้าวหน้า Tool ล้มเหลว หยุดเพราะงบหมด และทำงานสำเร็จก่อนตรวจ

→ ตรวจสอบความเข้าใจ

เมื่อ Tool คำนวณสำเร็จ แต่ Tool เวลาเชื่อมต่อไม่ได้ ระบบควรตอบส่วนใดได้ ส่วนใดต้องเว้นไว้ และสถานะรวมควรเป็นอะไร?

Loop ทำงานได้ก็ต่อเมื่อ Tool มีสัญญาณที่ชัด บทถัดไปจะลงรายละเอียดว่าชื่อ คำอธิบาย Arguments และ Result ต้องออกแบบอย่างไร

อ่านเพิ่มเติม [01] OpenAI: Function calling · [03] OpenAI: Evaluation best practices

อ่านโค้ดจริงและสัญญาของ API

เส้นทางสำคัญใน Lab 3

โค้ดส่วนนี้คัดเฉพาะจุดสำคัญจาก `run_agent` โดยเว้นส่วนการบันทึกข้อความและส่วนอื่นออก ใช้สำหรับอ่านหน้าที่ของแต่ละบรรทัด ไม่ใช่โปรแกรมสมบูรณ์ที่คัดลอกไปรันได้ [C1]

```
resp = llm.chat(messages=messages, tools=T00LS)
msg = resp.choices[0].message
if msg.tool_calls:
    # Other message-handling lines omitted here
    for call in msg.tool_calls:
        args = json.loads(call.function.arguments or "{}")
        result = dispatch(call.function.name, args)
```

บรรทัด `llm.chat` ส่งคำขอไปยังชั้นเรียกโมเดล `msg.tool_calls` คือจุดตรวจคำร้องขอ ส่วน `dispatch` คือทางเข้าสู่ฟังก์ชันจริง การอ่าน JSON สำเร็จไม่ได้เท่ากับตรวจ Schema หรือสิทธิ์ครบแล้ว โค้ดตัวอย่างจึงต้องอ่านพร้อมขอบเขตของมัน

ชื่อ Field ไม่ใช่ศัพท์กลางที่ใช้แทนกันได้

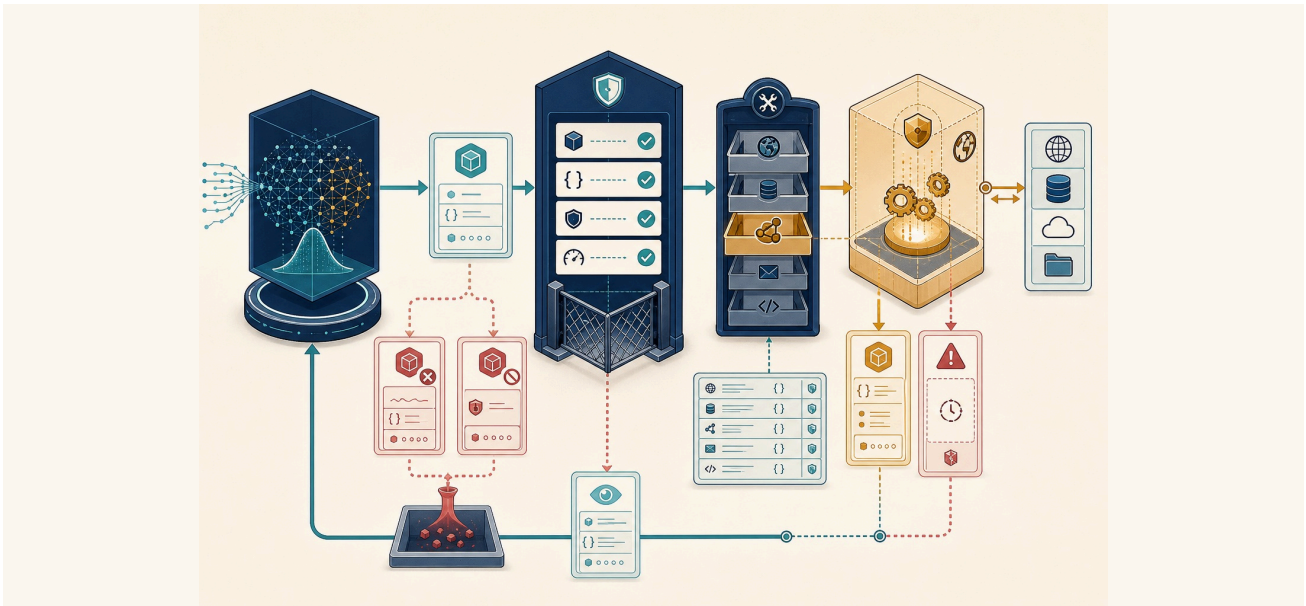
API / ตัวอย่าง	สัญญาหรือรายการที่ต้องอ่าน
Lab 3 / Chat Completions	<code>choices[0].message.tool_calls</code>
OpenAI Responses	รายการ <code>function_call</code> และผล <code>function_call_output</code> ที่ผูก <code>call_id</code>
Anthropic client tools	<code>tool_use</code> และ <code>tool_result</code> ที่ผูก <code>tool_use_id</code> รวมถึง <code>stop_reason</code>

ตารางนี้เทียบ Client/Function Tool เป็นหลัก เครื่องมือที่ผู้ให้บริการรันให้มีเส้นทางจัดการต่างกัน จึงไม่ควรใช้คำว่า `stop_reason` ของ API หนึ่งจะควบคุม Loop ของทุก API ได้ [O1, A7]

อ่านเพิ่มเติม [C1] Course source: Lab 3: [agent_loop.py](#) · [O1] OpenAI: [Function calling](#) · [A7] Anthropic: [Tool use with Claude](#)

บทที่ 4 - Tool Calling: สัญญาระหว่างโมเดลกับซอฟต์แวร์

ออกแบบความสามารถที่โมเดลเลือกใช้ได้ และ Runtime ตรวจสอบได้



ภาพแนวคิดจากหนังสือตัวอย่างของผู้เขียน ใช้อธิบายความสัมพันธ์ ไม่ใช่ Trace ของระบบที่รันจริง

AI Tool ไม่จำเป็นต้องมี AI อยู่ข้างใน

ฟังก์ชันอ่านเวลา คำนวณเลข หรืออ่านข้อมูล อาจเป็นโค้ดธรรมดา สิ่งที่ทำให้มันเป็น Tool ในระบบ Agent คือระบบประกาศความสามารถให้โมเดลรู้ และมี Runtime ที่รับคำร้องขอแล้วเรียกฟังก์ชันนั้นได้ ชื่อฟังก์ชันเพียงอย่างเดียวไม่ทำให้โมเดลรู้วิธีใช้ [01]

ในขั้นเรียน ตัวอย่าง Execute Query ช่วยให้เห็นสามส่วนสำคัญคือชื่อ Tool, Argument และ Description หนังสือเพิ่มอีกส่วนคือ Result Contract เพื่อให้ระบบรู้ว่าจะได้ผลแบบใดและใช้ตรวจสอบความสำเร็จอย่างไร

◆ หลักคิด

Tool Calling คือคำร้องขอ ส่วน Tool Execution คือเหตุการณ์ที่ Runtime ทำงานจริง

สัญญาส่วนของเครื่องมือ

ส่วน	คำถามที่ต้องตอบ	ตัวอย่างเชิงออกแบบ
Name	ความสามารถนี้คืออะไร	get_sales_by_year
Description	ควรใช้เมื่อใดและมีข้อจำกัดอะไร	อ่านยอดขายรวมตามปีที่ระบุ
Input contract	รับค่าแบบใดและช่วงใด	ปีเริ่ม ปีสิ้นสุด และหน่วยธุรกิจ
Result contract	คืนข้อมูลและข้อจำกัดอย่างไร	รายการปี ยอดรวม หน่วย และความครบ

Description คือส่วนหนึ่งของการออกแบบพฤติกรรม

คำอธิบายว่า “ใช้ Query ฐานข้อมูล” กว้างเกินไปสำหรับบางงาน ควรระบุความหมายของข้อมูล ช่วงเวลาที่รองรับ สิทธิ์ และกรณีที่ไม่มีข้อมูล หาก Tool หลายตัวชื่อคล้ายกัน ให้เขียนขอบเขตต่างกันชัดเจนเพื่อลดการเลือกผิด หลักนี้สอดคล้องกับแนวทางออกแบบ Tool ของ Anthropic [A3]

Docstring อาจถูก Framework นำไปสร้าง Description แต่ไม่ใช่ทุก Framework จะทำเหมือนกัน ต้องตรวจ Definition ที่ส่งถึงโมเดลจริง การเขียน Docstring ดีจึงยังต้องตามด้วยการตรวจ Schema และการใช้ Tool ใน Trace

Schema ถูก ไม่ได้แปลว่าการกระทำถูก

Strict Schema ช่วยให้อารมณ์ Arguments ตรงสัญญาที่กำหนด แต่ค่าที่มีชนิดถูกอาจยังผิดคน ผิดปี หรือเกินสิทธิ์ Runtime ต้องตรวจเงื่อนไขธุรกิจและ Authorization เพิ่ม เช่น year เป็นจำนวนเต็มได้ แต่ยังคงอยู่ในช่วงที่อนุญาต [O1]

อ่านเพิ่มเติม [O1] OpenAI: Function calling · [A3] Anthropic: Writing effective tools for AI agents

Dispatch คือจุดเปลี่ยนจากข้อความเป็นผลกระทบ

ใน Lab ชื่อ Tool ถูกจับคู่กับฟังก์ชันผ่าน dispatch จุดนี้จึงเป็น Boundary สำคัญ: โมเดลอาจเสนอชื่ออะไรก็ได้ แต่ Runtime ไม่ควรเรียกสิ่งที่ไม่ได้อนุญาต การสร้างข้อความโค้ดใหม่ก็ไม่ทำให้โค้ดนั้นรันได้ หากไม่มีความสามารถสำหรับการรันโค้ด

```
# Conceptual dispatch, not runnable lab code
call = parse_model_tool_request(output)
validate_schema(call.arguments)
require_allowed_tool(call.name)
require_authorized_user_and_target(call)
result = registry[call.name](call.arguments)
return result_with_call_id_and_status(result)
```

ออกแบบ Tool ให้ทำผิดได้ยาก

ถ้างานต้องการยอดขายรายปี Tool ที่รับปีและหน่วยธุรกิจอาจควบคุมง่ายกว่า Tool ที่รับ SQL อีสระ นี่เป็นทางเลือกเชิงออกแบบ ไม่ใช่กฎว่าทุกระบบห้ามใช้ SQL Tool ขอบเขตจริงต้องสัมพันธ์กับโจทย์ สิทธิฐานข้อมูล และความสามารถในการตรวจคำสั่ง

การตรวจเพียงว่ามีคำว่า SELECT หรือไม่มี DELETE ไม่ใช่มาตรการครบถ้วน เพราะรูปแบบคำสั่งและสิทธิ์ของระบบมีความซับซ้อน ควรจำกัดสิทธิ์ที่ฐานข้อมูลจริง แยกงานอ่านกับงานเขียน และตรวจสอบสัญญาของ Tool โดยไม่ฝากทุกอย่างไว้กับการจับคำใน String

! Boundary ที่ต้องบังคับ

โมเดลเสนอได้ แต่ Runtime ต้องเป็นผู้ตรวจว่าการกระทำนั้นอยู่ในสัญญาและสิทธิ์ที่ได้รับ

อ่านเพิ่มเติม [O1] OpenAI: Function calling · [O4] OpenAI: Guardrails and human review

ผลของ Tool ต้องมีความหมาย

ข้อมูลมากไม่ได้แปลว่าหลักฐานดี

ผลลัพธ์ที่มีทุก Column อาจเปลี่ยน Context และเปิดเผยข้อมูลที่ไม่จำเป็น ในทางกลับกัน การย่อมากเกินไป แค่ “พบข้อมูล” ทำให้ตรวจไม่ได้ว่าคำตอบคืออะไร Result ที่ดีควรให้ข้อมูลที่พอสำหรับงาน พร้อมสถานะ ความครบ และจุดอ้างอิงกลับ

ผลที่คลุมเครือ	ผลที่ตรวจได้มากขึ้น
success: true	บอกว่าขั้นใดสำเร็จ และมีผลลัพธ์หรือการเปลี่ยนแปลงใด
ไม่พบข้อมูล	ระบุขอบเขตที่ค้น และแยกไม่พบออกจากค้นไม่สำเร็จ
ยอดขาย = 100	ระบุปี หน่วย ขอบเขตองค์กร และแหล่งข้อมูล
Error	บอกประเภท ความสามารถในการลองใหม่ และสิ่งที่ยังไม่ได้ทำ

✓ Harness Design Check

- **Harness รับผิดชอบ:** ตรวจ Schema และสิทธิ์ เรียกฟังก์ชัน และจัด Result ให้ใช้งานต่อได้
- **Model ตัดสินใจ:** เลือก Tool และเสนอ Arguments จากโจทย์กับบริบท
- **Evidence ที่ต้องเก็บ:** Definition, Arguments, Call ID, สถานะ ผล และข้อจำกัด
- **Failure ที่ต้องทดสอบ:** Tool ชื่อผิด ค่าเกินขอบเขต ข้อมูลไม่ครบ ผลใหญ่เกิน และ Error ที่ถูกกลบ

→ ตรวจสอบความเข้าใจ

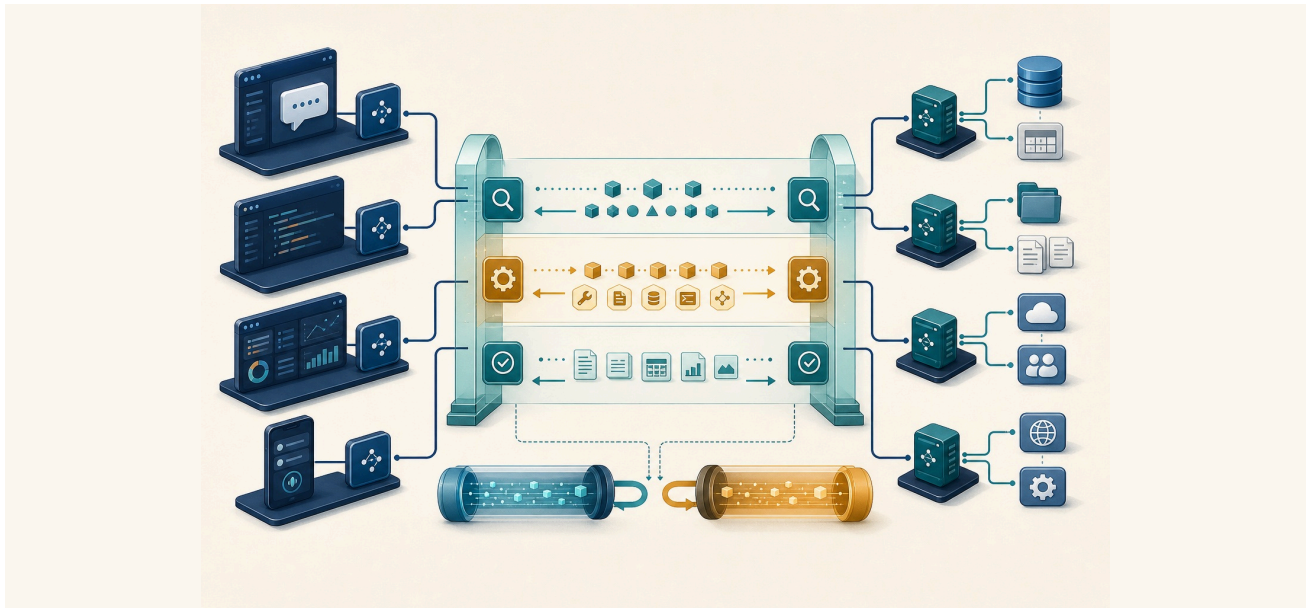
ถ้า Query สำเร็จแต่ได้ 5 แถวที่เป็นปีเดียวกันทั้งหมด เราควรถือว่าคำขอ “เปรียบเทียบ 5 ปี” สำเร็จหรือไม่?

เมื่อหลายระบบต้องใช้ Tool ร่วมกัน การประกาศและเรียกความสามารถผ่านมาตรฐานช่วยลดงานเชื่อมต่อ เฉพาะคุณ นี่คือบทบาทของ MCP ในบทความต่อไป

อ่านเพิ่มเติม [A3] Anthropic: Writing effective tools for AI agents · [O1] OpenAI: Function calling

บทที่ 5 - MCP: ภาษากลางระหว่าง Agent กับเครื่องมือ

แยกมาตรฐานการเชื่อมต่อออกจาก Framework และนโยบายความปลอดภัย



ภาพแนวคิดจากหนังสือตัวอย่างของผู้เขียน ใช้อธิบายความสัมพันธ์ ไม่ใช่ Trace ของระบบที่รันจริง

ลดการผูกเครื่องมือไว้กับแอปเดียว

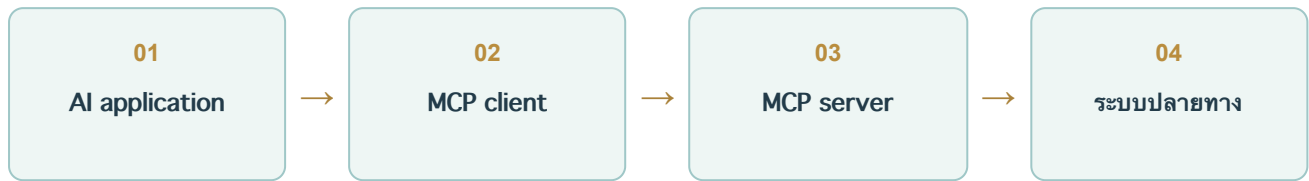
เมื่อหลายแอปต้องใช้ข้อมูลชุดเดียวกัน การเขียนการเชื่อมต่อแยกทุกคู่เพิ่มภาระดูแล MCP จัดรูปแบบการเชื่อมระบบ AI กับความสามารถภายนอกผ่านมาตรฐานร่วม ผู้พัฒนาจึงเปิดความสามารถฝั่ง Server และให้ Client ที่รองรับมาตรฐานเชื่อมต่อ [A6]

ในวันแรก ผู้สอนยกการแยก Tool ออกจาก Agent เป็นเหตุผลด้านการดูแลโค้ดและ Credential แนวคิดนี้มีประโยชน์ แต่การแยก Process หรือเครื่องไม่ได้ทำให้ปลอดภัยเอง ต้องกำหนดตัวตน สิทธิ์ และข้อมูลที่ส่งผ่านด้วย

◆ หลักคิด

MCP บอกวิธีคุยกัน ส่วนใครทำอะไรได้ยังเป็นหน้าที่ของระบบและนโยบาย

Host, Client และ Server



แยกบทบาทก่อนเลือกตำแหน่งติดตั้ง

แอปที่ผู้ใช้ทำงานด้วยเป็น Host ของประสบการณ์ใช้งาน ภายในมี Client ที่เชื่อมกับ Server ส่วน Server เปิดความสามารถที่ระบบเลือกให้เข้าถึง การแยกบทบาทเชิงตรรกะไม่ได้บังคับว่าทุกส่วนต้องอยู่คนละเครื่อง Server อาจอยู่ในเครื่องเดียวกันหรือเป็นบริการระยะไกลตามรูปแบบที่รองรับ

ส่วน	สิ่งที่ต้องออกแบบ
Host / แอป	บริบทของงาน การอนุญาต และการเลือกความสามารถ
Client	การเชื่อมต่อ การอ่านรายการ Tool และการส่งคำขอ
Server	สัญญา Tool ลิขสิทธิ์แหล่งข้อมูล และผลตอบกลับ
ระบบปลายทาง	สิทธิ์จริง ความถูกต้องของข้อมูล และข้อจำกัดการทำงาน

FastMCP เป็นทางเลือกในการพัฒนา

MCP เป็นมาตรฐาน ไม่ได้บังคับว่า Python ต้องใช้ FastMCP Framework ช่วยลดงานเขียนรายละเอียด แต่ความสอดคล้องกับมาตรฐานต้องดูพฤติกรรมบน Protocol การเอา import ออกแล้วโปรแกรมพึ่งพิลิซจน์เพียงว่าไค้ด้นั้นพึ่ง Library ดังกล่าว ไม่ได้พิสูจน์ว่ามาตรฐานบังคับ Library นั้น

♦ ถ้อยคำที่ปรับให้แม่นยำ

แยก “ข้อกำหนดของมาตรฐาน” ออกจาก “วิธีที่ตัวอย่างนี้เลือกใช้” แล้วผู้เรียนจะเปลี่ยนภาษาและ Framework ได้โดยยังรักษาสัญญาการเชื่อมต่อ

อ่านเพิ่มเติม [A6] Anthropic: Introducing the Model Context Protocol · [O5] OpenAI: Remote MCP servers

ค้นพบ Tool และเรียกใช้ Tool เป็นคนละขั้น

รายการความสามารถไม่ใช่ผลการทำงาน

ก่อนใช้ Tool แอปต้องทราบว่ามีความสามารถใดและรับค่าอย่างไร หลังจากเลือกแล้วจึงส่งคำร้องขอพร้อม Arguments ใน OpenAI Responses API การเชื่อม Remote MCP มีรายการผลที่แยกการนำเข้ารายการ Tool ออกจากการเรียก Tool จริง [O5]

สิ่งที่ตรวจ	คำถามที่ตอบได้
รายการ Tool	ระบบมองเห็นชื่อ คำอธิบาย และ Schema อะไร
Tool Call	โมเดลร้องขอความสามารถใดด้วยค่าอะไร
ผลตอบกลับ	Server ทำอะไรสำเร็จหรือไม่สำเร็จ
ผลปลายทาง	ข้อมูลหรือสถานะจริงตรงตามที่รายงานหรือไม่

การเพิ่ม Tool ฝั่ง Server ไม่ได้แปลว่าทุก Client จะเห็นทันทีเสมอไป บางระบบเก็บรายการไว้ใน Context หรือ Cache จึงต้องตรวจกลไก Refresh และการรองรับของ Client การออกแบบเปลี่ยนแปลงต้องคำนึงถึงเวอร์ชัน Schema และผู้ใช้เดิม

แยก Credential ได้ แต่ต้องตรวจสอบสิทธิ์ด้วย

การเก็บ Credential ฐานข้อมูลไว้ฝั่ง Server ลดความจำเป็นที่ Agent จะถือ Secret โดยตรง แต่ Server ยังต้องบังคับสิทธิ์ของผู้เรียก แยกขอบเขตข้อมูล และหลีกเลี่ยงคืน Secret ผ่านผล Tool OpenAI ระบุความเสี่ยงของ Server ที่ไม่น่าเชื่อถือและการส่งข้อมูลออกผ่าน Tool ไวชัดเจน [O5]

! ขอบเขตความเข้ากันได้

การมี MCP URL ไม่ได้ยืนยันว่าใช้กับทุกแอปได้ ต้องตรวจ Transport, Authentication และความสามารถที่แอปนั้นรองรับ

อ่านเพิ่มเติม [O5] OpenAI: Remote MCP servers · [A6] Anthropic: Introducing the Model Context Protocol

เลือก MCP เมื่อประโยชน์คุ้มภาระ

Custom Tool ยังมีที่ยืน

Tool ที่อยู่ในโค้ดแอปโดยตรงไม่ได้ด้อยคุณภาพเพียงเพราะไม่ได้ใช้ MCP ถ้าแอปเดียวใช้ความสามารถเล็กน้อย การเรียกฟังก์ชันตรงอาจเรียบง่ายกว่า MCP มีประโยชน์เมื่อจำเป็นต้องแชร์ความสามารถ แยกการดูแล หรือเชื่อมกับระบบที่รองรับมาตรฐานนั้นอยู่แล้ว

สำหรับระบบแบบลากวาง ต้องแยกตัว Agent, MCP Client และ MCP Server ให้ออก การใช้แพลตฟอร์มดังกล่าวไม่ได้ทำให้ระบบใช้จริงไม่ได้โดยอัตโนมัติ ควรตรวจสอบสิทธิ์ Error Handling, Observability และขอบเขตการ Deploy จากพฤติกรรมจริงของระบบ

✓ Harness Design Check

- **Harness** รับผิดชอบ: เลือก Tool ที่เปิดให้ใช้ ตรวจสอบ Server และควบคุมข้อมูลที่ส่งออก
- **Model** ตัดสินใจ: เลือกความสามารถที่ประกาศและเสนอ Arguments
- **Evidence** ที่ต้องเก็บ: รายการ Tool, Server identity, Request/Result และผลอนุมัติ
- **Failure** ที่ต้องทดสอบ: Server ไม่พร้อม Schema เปลี่ยน Tool ไม่ปรากฏ และผลที่มีข้อมูลอ่อนไหว

→ ตรวจสอบความเข้าใจ

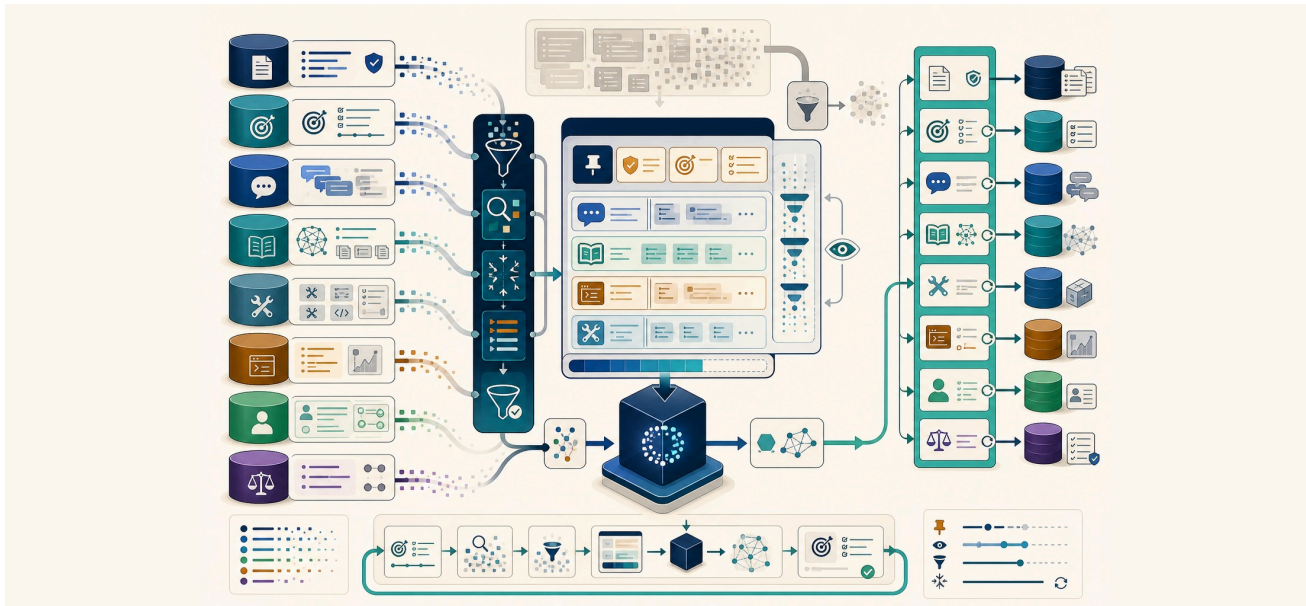
ถ้ามี Agent 20 ตัวใช้ Tool เดียวกัน คุณจะเปลี่ยน Schema โดยไม่ทำให้ผู้ใช้เดิมเสียอย่างไร และจะรู้ได้อย่างไรว่า Client เห็นเวอร์ชันใหม่แล้ว?

MCP ทำให้เชื่อมข้อมูลได้สะดวกขึ้น แต่ข้อมูลที่ดึงมาจะกลายเป็น Context ซึ่งมีข้อจำกัด บทถัดไปจึงเน้นสิ่งที่ควรส่งให้โมเดลในแต่ละรอบ

อ่านเพิ่มเติม [O5] OpenAI: Remote MCP servers · [A6] Anthropic: Introducing the Model Context Protocol

บทที่ 6 - Context และความต่อเนื่องของงาน

ให้ข้อมูลพอสำหรับก้าวถัดไป โดยรักษาข้อเท็จจริงที่สำคัญ



ภาพแนวคิดจากหนังสือตัวอย่างของผู้เขียน ใช้อธิบายความสัมพันธ์ ไม่ใช่ Trace ของระบบที่รันจริง

โมเดลเห็นสิ่งที่ระบบจัดให้ในรอบนั้น

Chat History และ Tool Result เติบโตตามการใช้งาน ในวันแรก ผู้สอนใช้การสะสมประวัติทั้งปีเป็นตัวอย่างว่าเราไม่ควรใส่ทุกอย่างกลับเข้าโมเดลโดยไร้การเลือก Context Engineering จึงเริ่มจากคำถามว่า “รอบนี้ต้องรู้ข้อมูลอะไรเพื่อทำงานต่อให้ถูก”

Context Window เป็นขอบเขตของโมเดลหรือบริการ ไม่ใช่สัญญาว่าทุกข้อมูลภายในขอบเขตจะถูกใช้ได้แม่นยำเท่ากัน การเลือกข้อมูลที่ตรงงานช่วยให้ตรวจเหตุผลของระบบง่ายขึ้น และลดข้อมูลที่ไม่จำเป็น [A2]

◆ หลักคิด

รักษาเป้าหมาย ข้อจำกัด และหลักฐานที่มีผลต่อการตัดสินใจ ก่อนเก็บข้อความทุกประโยค

Context, History, State และ Memory

คำ	ความหมายที่ใช้ในเล่ม	ตัวอย่าง
Context	ข้อมูลที่ทำให้โมเดลใช้ในการเรียกครั้งนี้	คำถามล่าสุด ผล Tool ที่เกี่ยวข้อง
History	บันทึกการสนทนาและเหตุการณ์ที่ผ่านมา	ข้อความและ Tool Call หลายรอบ
Task State	สถานะที่จำเป็นต่อภารกิจ	ค้นปีใดแล้ว ปีใดยังขาด
Memory	ข้อมูลที่จัดเก็บไว้เพื่อเรียกใช้อีก	ข้อจำกัดที่ผู้ใช้ยืนยันและควรจำต่อ

Stateless ไม่ได้หมายความว่าแอปไม่มีทางจำ

คำอธิบาย “โมเดลไม่มี Memory” ในวันแรกควรอ่านในความหมายว่า การเรียกโมเดลครั้งใหม่ไม่ได้รู้ประวัติครั้งก่อนเองหากไม่มีการจัดข้อมูลให้ แอปอาจเก็บ History และส่งซ้ำ หรือใช้บริการจัดการ Conversation State ความต่อเนื่องจึงเกิดจากระบบที่จัดเก็บและนำข้อมูลกลับมา [O2]

ไม่ควรขยายความนี้ว่า โมเดลไม่มีความรู้ที่เรียนมา หรือบริการทุกชนิดห้ามเก็บสถานะ เพราะเป็นคนละประเด็น ความรู้จากการฝึก การจดจำบทสนทนา และสถานะของงานต้องแยกกันในการออกแบบ

State ที่เล็กแต่มีประโยชน์

สำหรับงานยอดขายหลายปี State อาจเก็บช่วงปีที่ผู้ใช้ยืนยัน แหล่งข้อมูล ปีที่ตรวจแล้ว และรายการที่ยังขาด ไม่จำเป็นต้องคัดลอกผลดิบทุกแถวไปทุกครั้ง แต่ต้องเก็บทางย้อนกลับไปตรวจหลักฐานได้

อ่านเพิ่มเติม [O2] OpenAI: Conversation state · [A2] Anthropic: Effective context engineering for AI agents

บริบทที่โตขึ้นต้องมึนโยบาย

เลือกก่อน ย่อเมื่อจำเป็น และรักษาส่่งที่ห้ามหาย

วิธีจัดการอาจเริ่มจากจำกัดผล Tool ให้ตรงคำถาม ดึงข้อมูลเฉพาะเมื่อจำเป็น และย่อเหตุการณ์ที่ไม่ต้องใช้รายละเอียดแล้ว การย่อเป็นการเปลี่ยนหลักฐานที่โมเดลเห็น จึงต้องตรวจว่าข้อจำกัดหรือผลที่ยังไม่แน่นอนไม่ถูกเปลี่ยนเป็นข้อสรุป

ควรรักษา	ลดหรือย้ายออกได้เมื่อมีทางอ้างกลับ
เป้าหมายและขอบเขตที่ผู้ใช้ยืนยัน	คำอธิบายซ้ำและข้อความทั่วไป
การอนุมัติและเงื่อนไขห้ามทำ	ผล Tool เก่าที่ไม่เกี่ยวกับรอบปัจจุบัน
ข้อผิดพลาดที่ยังแก้ไขไม่สำเร็จ	ข้อมูลดิบจำนวนมากที่สรุปอย่างตรงใจได้
รหัสหรือแหล่งอ้างอิงหลักฐาน	ประวัติที่ไม่เปลี่ยนการตัดสินใจแล้ว

ตัวอย่างเช่น “ยังไม่ได้ตรวจปีสุดท้าย” ต้องไม่ถูกย่อเป็น “ตรวจสอบข้อมูลครบแล้ว” และ “ผู้ใช้อนุญาตให้อ่าน” ต้องไม่กลายเป็น “อนุญาตให้แก้ไข” ความผิดพลาดแบบนี้กระทบพฤติกรรมของ Agent โดยตรงแม้ข้อความสรุปดูอ่านง่าย

ข้อมูลภายนอกเป็นหลักฐาน ไม่ใช่ผู้กำหนดกฎ

เนื้อหาที่อ่านจากเอกสารหรือ Tool อาจมีข้อความลักษณะคำสั่ง ระบบต้องรักษาขอบเขตระหว่างคำสั่งของผู้ใช้กับข้อมูลที่ค้นพบ การนำผล Tool กลับเข้า Context ไม่ได้ทำให้ข้อความนั้นมีสิทธิ์เปลี่ยนนโยบายของแอป

→ **โจทย์ตรวจความต่อเนื่อง**

ให้ทดสอบหลังย่อ Context ว่า Agent ยังรู้สิ่งที่ทำแล้ว สิ่งที่ยังขาด และสิ่งที่ไม่ได้รับอนุญาตหรือไม่

อ่านเพิ่มเติม [A2] Anthropic: Effective context engineering for AI agents · [O2] OpenAI: Conversation state · [O5] OpenAI: Remote MCP servers

Context ที่ดีช่วยให้ Loop เดินต่ออย่างมีหลักฐาน

ต้นทุนของข้อมูลที่ไม่เกี่ยวข้อง

ทุกครั้งที่ส่งบริบทใหม่ ระบบมีทั้งต้นทุน Token เวลา และโอกาสที่ข้อมูลเก่าจะรบกวนงานปัจจุบัน การเพิ่ม Context อย่างเดียวอาจไม่แก้คำตอบผิด ถ้ารากปัญหาคือเลือกข้อมูลผิดแหล่งหรือปะปนข้อสันนิษฐานกับผลที่ตรวจแล้ว

ควรบันทึกว่าบริบทมาจากไหน ถูกเลือกหรือย่ออย่างไร และเวอร์ชันใดถูกส่งเข้าโมเดล การรู้เพียงคำตอบสุดท้ายไม่พอแยกว่าปัญหาเกิดจาก Tool, การคัดข้อมูล หรือการสร้างคำตอบ

✓ Harness Design Check

- **Harness รับผิดชอบ:** เลือกบริบท เก็บ State และรักษาข้อจำกัดซ้ำมรอบ
- **Model ตัดสินใจ:** ใช้ข้อมูลที่มีเพื่อตัดสินใจขั้นถัดไป หรือระบุข้อมูลที่ยังขาด
- **Evidence ที่ต้องเก็บ:** แหล่งข้อมูล เวอร์ชัน State และการเปลี่ยนบริบทที่สำคัญ
- **Failure ที่ต้องทดสอบ:** สัมข้อห้าม สรุปความไม่แน่นอนเป็นข้อเท็จจริง และส่งข้อมูลเก่าแทนปัจจุบัน

→ ตรวจสอบความเข้าใจ

ถ้าต้องย่อบทสนทนาเหลือเพียงหนึ่งหน้า จะเก็บอะไรเพื่อให้ระบบทำงานต่อโดยไม่ทำซ้ำและไม่ขยายสิทธิ์? อธิบายเหตุผลของแต่ละรายการ

การจัดข้อมูลช่วยให้โมเดลตัดสินใจดีขึ้น แต่กฎสำคัญบางอย่างต้องบังคับใน Runtime บทถัดไปจึงนำ Hook, Guardrail และ Sandbox มาแยกหน้าที่ให้ชัด

อ่านเพิ่มเติม [O2] OpenAI: Conversation state · [A2] Anthropic: Effective context engineering for AI agents

บทที่ 7 - Hooks, Guardrails และขอบเขตการทำงาน

กำหนดว่ากฎทำงานตรงไหน ใครตัดสิน และอะไรถูกบังคับจริง



อุปมาด้านตรวจในชั้นเรียน

ผู้สอนเปรียบ Hook เป็นตำแหน่งตั้งด่าน และ Middleware เป็นกฎที่ทำงานตรงด่าน อุปมาช่วยแยกแยะแกนของ Loop ออกจากกติกาที่เปลี่ยนบ่อย เช่น ตรวจสอบเครื่องมือก่อนรัน ปิดบังข้อมูลก่อนส่งกลับ หรือบันทึกเหตุการณ์เพื่อดูย้อนหลัง

ในระดับซอฟต์แวร์ Hook เป็นจุดที่ระบบเปิดให้เรียก Logic เพิ่ม ไม่จำเป็นต้องทำหน้าที่ Block เสมอไป ส่วน Guardrail เป็นการตรวจหรือควบคุมพฤติกรรม และอาจวางผ่าน Hook, Tool Wrapper, สิทธิ์ของบริการ หรือกลไกอื่น จึงไม่ควรใช้ความสัมพันธ์ว่า “Guardrail ทั้งหมดเป็นส่วนย่อยของ Hook” เป็นนิยามสากล [O4, A4]

กฎใน Prompt กับกฎใน Runtime

คำสั่งให้โมเดลระวังข้อมูลลับช่วยขึ้นการตัดสินใจ แต่หากข้อมูลนั้นห้ามออกจริง ต้องมีจุดตรวจและสิทธิ์ที่บังคับได้ ด้วย วิธีนี้แยกสิ่งที่หวังให้โมเดลทำออกจากสิ่งที่ระบบยอมให้เกิด

♦ โครงสร้างของการควบคุม

กำหนดเหตุการณ์ → ตรวจสอบข้อมูล → คัดค้านตัดสิน → ให้ Runtime บังคับคำตัดสิน

อ่านเพิ่มเติม [O4] OpenAI: Guardrails and human review · [A4] Anthropic: Hooks reference - Claude Code

ก่อนและหลังการกระทำให้ผลต่างกัน

ตำแหน่ง	ตรวจอะไร	ข้อจำกัด
ก่อนเรียกโมเดล	ข้อมูลที่กำลังส่งออก ขนาด และความพร้อม	ต้องบังคับก่อนส่งจริง
ก่อนเรียก Tool	ชื่อ ค่า สิทธิ์ และผลกระทบ	จุดหลักในการป้องกัน Action
หลัง Tool	รูปแบบผล ความครบ และข้อมูลลับ	ไม่ย้อนผลกระทบที่เกิดขึ้นแล้ว
ก่อนตอบหรือจบ	คำตอบและหลักฐานตรงเป้าหมายหรือไม่	ต้องมีทางหยุดเมื่อไม่ผ่าน

Post-tool ไม่สามารถย้อนการลบที่เกิดขึ้นแล้ว

ถ้า Tool ลบข้อมูลสำเร็จแล้ว Hook หลัง Tool ตรวจพบว่าไม่ควรทำ การตรวจนั้นช่วยรายงานหรือเริ่มกระบวนการแก้คืนได้ แต่ไม่ได้ทำให้การลบไม่เคยเกิดขึ้น กฎที่ต้องป้องกันผลกระทบต้องอยู่ก่อน Execution หรืออยู่ในระบบปลายทางที่บังคับการกระทำนั้น [A4]

ชื่อเหตุการณ์ใน Lab เช่น pre_llm และ pre_tool เป็นชื่อที่ผู้พัฒนา Lab ตั้ง ไม่ใช่สัญญาว่า Claude Code หรือ OpenAI SDK มี Event และความหมายตรงกันทุกตัว ก่อนใช้ Framework จริงต้องตรวจว่าคำตัดสินจาก Hook ถูกอ่านและมีผลอย่างไร

Human review ต้องเกิดก่อนการตัดสินที่เสี่ยง

การขออนุมัติควรแสดงเป้าหมาย Arguments และผลกระทบที่เสนอ ให้ผู้มีสิทธิ์ตัดสินกับการกระทำที่ชัดเจน ไม่ใช่ขออนุญาตกว้าง ๆ แล้วตีความเป็นสิทธิ์ทุกครั้ง OpenAI แยกการตรวจอัตโนมัติออกจากการหยุดรออนุมัติไว้โดยตรง [O4]

อ่านเพิ่มเติม [A4] Anthropic: Hooks reference - Claude Code · [O4] OpenAI: Guardrails and human review

HookResult และ HookManager ใน Lab

สไลด์หน้า 33–35 เชื่อม Lab 3 กับ Lab 4 โค้ดที่ตรวจอ่าน ณ Commit [9af02acc6861](#) ยืนยันว่า Lab 4 นำ TOOLS, SYSTEM และ dispatch จาก Lab 3 มาใช้ แล้วเขียน Loop ที่เรียก Hook รอบจุดสำคัญ ข้อมูลต่อไปนี้นำมาจากการอ่าน Source Code ไม่ใช่ผลทดลองรัน [C1–C3]

ส่วน	หน้าที่ที่พบในโค้ด
HookResult	เก็บ decision, reason, data และ additional_context
HookManager.register	ลงทะเบียน Function ตาม Event และตัวกรองชื่อ Tool
HookManager.run	เรียกตามลำดับ เมื่อเจอ deny จะหยุด Chain และคืนผล
modify	รวมข้อมูลที่แก้แล้วให้ Hook ถัดไปเห็น ก่อนคืน Payload ล่าสุด

ตัวอย่างสี่ตัวสอนคนละหน้าที่

audit_log_hook บันทึกเหตุการณ์ **guard_calculate_hook** ตรวจสอบความยาวนิพจน์และตัดช่องว่าง **redact_secrets_hook** ปิดบังข้อความที่ตรง Pattern และ **require_number_on_stop_hook** ตรวจสอบว่าคำตอบมีตัวเลขเมื่อคำถามดูเหมือนโจทย์คำนวณ

ลำดับ post_tool วาง Redaction ก่อน Audit ทำให้ Log ของผล Tool เห็นค่าที่ผ่านการปิดบังแล้ว แต่ลำดับที่ดีเพียงจุดเดียวไม่ได้พิสูจน์ว่า Secret ไม่มีทางเข้าระบบผ่านทางอื่น เช่น Arguments, Prompt หรือช่องทาง Log อื่น

! อ่านให้ถึงผู้ใช้ผลลัพธ์

HookResult เป็นข้อมูลของคำตัดสิน ความเข้มของกฎจริงขึ้นกับว่า Loop อ่านและบังคับค่าที่คืนมาหรือไม่

อ่านเพิ่มเติม [C2] Course source: HookManager and HookResult · [C3] Course source: Lab 4: agent_loop_hooks.py · [S1] Aekanun Thongtae: PART 1: Harness & Agent Loop (37 slides)

ข้อจำกัดของตัวอย่างต้องมองเห็น

มี Hook ไม่ได้แปลว่าบังคับทุก Event แล้ว

จากการอ่าน Lab 4 เวอร์ชันที่อ้างอิง pre_llm นำ additional_context มาใช้ แต่ไม่ได้ใช้ decision เพื่อปฏิเสธการเรียกโมเดล ส่วนค่าที่คืนจาก post_llm ไม่ได้ถูกนำไปตัดสินใจเส้นทางต่อ จึงไม่ควรอธิบายว่าการเพิ่ม Hook ที่คืน deny ในสองตำแหน่งนี้จะหยุดการทำงานได้ทันที [C3]

Stop Hook ถูกออกแบบให้ขอคำตอบใหม่ได้หนึ่งครั้ง เมื่อใช้สิทธิ์ Retry แล้ว โค้ดอาจคืนคำตอบแม้ Hook ยังคืน deny จุดนี้เป็นกติกาตัวอย่างเพื่อกันวนไม่จบ ไม่ใช่ Hard Gate ที่รับประกันว่าคำตอบสุดท้ายผ่านทุกกฎ

เกณฑ์ง่ายเหมาะกับการสอน แต่มีขอบเขต

ตัวอย่าง	สิ่งที่ตรวจได้	สิ่งที่ยังรับประกันไม่ได้
ความยาวนิพจน์ ≤ 40	จำนวนตัวอักษรอยู่ในเกณฑ์	ต้นทุนประมวลผลหรือความปลอดภัยครบทุกกรณี
Pattern คล้าย Secret	ข้อความตรงรูปแบบที่ระบุ	Secret ทุกชนิดถูกปิดบัง
คำตอบมีตัวเลข	พบ Digit ในคำตอบ	คำนวณถูกหรือใช้ Tool Result จริง

! ข้อจำกัดที่ต้องคงไว้

อย่านำเกณฑ์ “มีตัวเลข” ไปเป็นเกณฑ์ Correctness และอย่านำ Pattern เดียวไปอ้างว่าป้องกันข้อมูลลับรั่วได้ทั้งหมด

เมื่อพัฒนาต่อ ให้กำหนดว่าจุดใดเป็นเพียง Feedback จุดใดเป็น Hard Gate และกรณีตรวจไม่ผ่านจะจบด้วยสถานะใด หนังสือไม่ได้แก้โค้ดหรือสไลด์ต้นฉบับ

อ่านเพิ่มเติม [C3] Course source: Lab 4: agent_loop_hooks.py · [O4] OpenAI: Guardrails and human review

Sandbox และสิทธิ์เป็นขอบเขตอีกชั้น

แยกความสามารถที่รันโค้ดออกจากการทำงานของรัน

การเพิ่ม Tool ที่รันโค้ดได้ทำให้ Agent มีความสามารถและพื้นที่ผลกระทบมากขึ้น ส่วน Sandbox มีหน้าที่จำกัดว่าสิ่งที่รันเข้าถึงไฟล์ เครือข่าย หรือทรัพยากรอะไรได้ จึงไม่ควรกล่าวว่า Sandbox เองทำให้ควบคุมได้น้อยลงโดยไม่มีเงื่อนไข Anthropic อธิบายการแยกขอบเขตไฟล์กับเครือข่ายเป็นกลไกสำคัญ [A5]

การมี Container, Runtime หรือ File System อย่างใดอย่างหนึ่งยังไม่พอให้สรุปว่าปลอดภัย ต้องตรวจสอบสิทธิ์ Mount, Credential, การเชื่อมต่อ และข้อจำกัดการทำงานจริง ขอบเขตที่ให้ควรพอทำงานที่ได้รับมอบหมายโดยไม่เปิดข้อมูลหรือระบบอื่นเกินจำเป็น

✓ Harness Design Check

- **Harness รับผิดชอบ:** กำหนด Event บังคับคำตัดสิน และจำกัดสิทธิ์ที่ Runtime/ปลายทาง
- **Model ตัดสินใจ:** เสนอการกระทำภายในขอบเขต หรือรายงานว่าต้องการสิทธิ์เพิ่ม
- **Evidence ที่ต้องเก็บ:** กฎที่ใช้ Payload หลังปรับ คำตัดสิน และผลการอนุมัติ
- **Failure ที่ต้องทดสอบ:** Hook ถูกเรียกแต่ผลถูกละเลย ตรวจสอบซ้ำเกินไป และ Log เก็บข้อมูลก่อนปิดบัง

→ ตรวจสอบความเข้าใจ

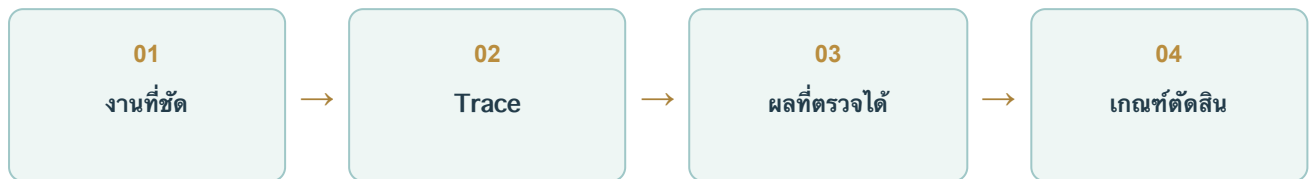
ถ้า Hook บอก deny แต่ Loop ยังเรียก Tool ต่อ ความผิดอยู่ที่ตัวโมเดล ตัว Hook หรือการเชื่อม Hook กับ Runtime? ต้องดูหลักฐานอะไรเพื่อแยก?

บทถัดไปจะใช้ Trace และกรณีล้มเหลวเหล่านี้สร้างเกณฑ์ประเมิน เพื่อให้การเปลี่ยน Harness มีหลักฐานรองรับ

อ่านเพิ่มเติม [A5] Anthropic: Making Claude Code more secure and autonomous with sandboxing · [O4] OpenAI: Guardrails and human review · [C3] Course source: Lab 4: agent_loop_hooks.py

บทที่ 8 - Evidence และการประเมิน Harness

วัดสิ่งที่ระบบทำจริง และตรวจว่าการแก้ไขทำให้ดีขึ้นหรือไม่



คำตอบดีครั้งเดียวไม่พอ

วันแรกใช้กราฟคะแนนแต่ละ Commit ชวนให้คิดเรื่อง Regression Testing แก่นสำคัญคือทุกครั้งที่เราแก้ Prompt, Tool หรือ Loop ต้องตรวจว่าพฤติกรรมเดิมเสียหรือไม่ แต่เส้นคะแนนที่นิ่งอย่างเดียวไม่พอ เพราะระบบอาจผิดอย่างสม่ำเสมอ ต้องพิจารณาระดับคุณภาพและความรุนแรงของข้อผิดพลาดด้วย

OpenAI แนะนำให้กำหนดเป้าหมายของการประเมิน ใช้ชุดงานที่ตรงกับการใช้งานจริง และติดตามผลต่อเนื่อง การตรวจเชิงตัวเลขควรใช้ร่วมกับการพิจารณาของมนุษย์ที่เข้าใจงาน [O3]

ประเมิน Model และ Harness ที่ทำงานร่วมกัน

ถ้าคำตอบผิด อาจเกิดจากดึงข้อมูลผิด ย่อ Context จนขาดรายละเอียด เรียก Tool ผิด หรือสร้างคำตอบไม่ตรงหลักฐาน การให้คะแนนคำตอบสุดท้ายอย่างเดียวยกตำแหน่งปัญหาไม่ได้ ต้องเก็บเหตุการณ์ระหว่างทางพอให้ตรวจสอบสาเหตุ

◆ หลักการทดลอง

เริ่มจากกรณีล้มเหลวจริง เก็บผลก่อนแก้ แล้วเปลี่ยนตัวแปรที่เชื่อมกับสาเหตุนั้น

อ่านเพิ่มเติม [O3] OpenAI: Evaluation best practices

Correctness กับ Faithfulness ตอบคนละคำถาม

มิติ	คำถามหลัก	หลักฐานอ้างอิง
Correctness	คำตอบถูกต้องตามโจทย์และข้อเท็จจริงหรือไม่	ข้อมูลที่เชื่อถือได้หรือคำตอบอ้างอิง
Faithfulness	ข้ออ้างในคำตอบมีหลักฐานที่ให้การรองรับหรือไม่	Context และ Tool Result ที่ระบบเห็น
Coverage / Recall	ครอบคลุมรายการที่ต้องพบอย่างน้อยเพียงใด	ชุดรายการที่คาดหวังและนิยามการนับ
Process compliance	ใช้เครื่องมือและสิทธิ์ตามกติกาหรือไม่	Trace และนโยบายของงาน

ในเล่มนี้ Ground หมายถึงหลักฐานที่ใช้ประกอบคำตอบ ส่วน Ground Truth คือข้อมูลอ้างอิงที่ยอมรับให้ใช้ตัดสินตามโจทย์ อย่างไรก็ตาม Tool Result ทุกชิ้นเป็น Ground Truth โดยอัตโนมัติ เพราะ Tool อาจค้นผิดแหล่ง ดึงข้อมูลเก่า หรือคืนผลไม่ครบ

ตัวอย่างที่แยกความถูกต้องออกจากการรองรับ

สมมติ Tool ดึงยอดขายผิดปีแล้ว โมเดลถ่ายทอดตัวเลขนั้นตรงทุกตัว คำตอบอาจสอดคล้องกับผล Tool แต่ยังไม่ถูกต้องตามโจทย์ ในทางกลับกัน โมเดลอาจเดาตัวเลขตรงโดยบังเอิญ แต่ไม่มีหลักฐานรองรับ จึงไม่ควรถือว่ากระบวนการน่าเชื่อถือ

! ขอบเขตของตัวอย่าง

นิยามมิติและหน่วยนับก่อนให้คะแนน หนังสือใช้ตัวอย่างเพื่ออธิบายเกณฑ์ ไม่ได้รายงานผลประเมินระบบหรือผู้เรียนที่ทดลองแล้ว

อ่านเพิ่มเติม [O3] OpenAI: Evaluation best practices

กรณีห่าปี: ตรวจสอบความหมายก่อนนับแถว

ห่าแถวไม่จำเป็นต้องเป็นห่าปี

คำขอให้เปรียบเทียบยอดขายห่าปีต้องนิยามช่วงเวลา หน่วย และขอบเขตองค์กรก่อน ถ้า Query คินห่าแถว แต่เป็นห่าสาขาในปีเดียว เกณฑ์นับ Row จะผ่านทั้งที่โจทย์ไม่สำเร็จ หรือถ้าระบบคินหนึ่งแถวที่มีห่าคอลัมน์ปี การตรวจว่าต้องมีอย่างน้อยห่าแถวก็อาจปฏิเสธข้อมูลที่ครบแล้ว

เงื่อนไขที่ควรตรวจ	ตัวอย่างคำถาม
ช่วงเวลา	เป็นห่าปีใด ปีปฏิทินหรือปีบัญชี
Coverage	ครอบคลุมปีที่ต้องการครบหรือยัง
หน่วยและขอบเขต	สกุลเงินและหน่วยธุรกิจตรงกันหรือไม่
ความหมายของค่า	ศูนย์ยอดขายต่างจากไม่มีข้อมูลหรือไม่
ความครบของคำตอบ	สรุปเปรียบเทียบตรงกับผลที่ตรวจแล้วหรือไม่

หากนิยามชุดอ้างอิงเป็นห่าปีที่ต้องพบ และระบบพบสามปีที่ถูกต้องไม่ซ้ำกัน จึงคำนวณ Coverage แบบง่ายได้ $3/5 = 60\%$ แต่ตัวเลขนี้ไม่ได้ประเมินความถูกต้องของยอดรวม หรือ Faithfulness ของคำอธิบายทั้งหมด ต้องแยกเกณฑ์เหล่านั้นอีกชั้น

♦ ตัวอย่างเชิงอธิบาย

ตัวเลข 3/5 เป็นตัวอย่างคำนวณตามนิยามที่กำหนด ไม่ใช่ผลวัดจากข้อมูลจริงในชั้นเรียน

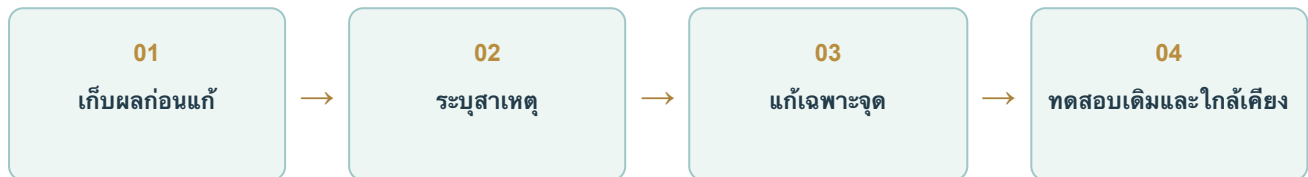
กฎที่ดีจึงสัมพันธ์กับความหมายของงาน ไม่ได้เลือกสิ่งที่เขียน if ได้ง่ายที่สุดแล้วเรียกว่าเป็นหลักฐานความสำเร็จ

อ่านเพิ่มเติม [03] OpenAI: Evaluation best practices

Regression ที่เชื่อมกับสาเหตุ

เปลี่ยนหนึ่งสาเหตุ แล้วกลับไปทดสอบจุดเดิม

เมื่อพบความล้มเหลว ให้เก็บคำขอ การตั้งค่า Trace และผลที่เห็นก่อนแก้ ตรวจสอบว่าจุดใดทำให้ผิด แล้วปรับตัวแปรที่เกี่ยวข้อง เช่น หาก Description คลุมเครือทำให้เลือก Tool ผิด ให้แก้คำอธิบายและรันกรณีเดิมก่อน เปลี่ยนโมเดลหรือโครงสร้าง Loop พร้อมกัน



หลังกรณีเดิมผ่าน ให้ตรวจกรณีใกล้เคียงและชุดกว้างขึ้น รวมกรณีที่ควรปฏิเสธหรือหยุดด้วย การประเมินที่ตรวจเฉพาะคำตอบสำเร็จอาจไม่เห็นพฤติกรรมใช้สิทธิ์เกินหรือซ่อน Error

✓ Harness Design Check

- Harness รับผิดชอบ: เก็บเงื่อนไขทดลอง Trace และกฎปล่อยงานที่อิงผล
- Model ตัดสินใจ: เป็นส่วนหนึ่งของระบบที่ถูกตรวจ ไม่ใช่ผู้รับรองความสำเร็จเอง
- Evidence ที่ต้องเก็บ: Baseline, ผลหลังแก้ รุ่น โมเดล เวอร์ชันโค้ด และผู้ประเมิน
- Failure ที่ต้องทดสอบ: Regression, หลักฐานไม่ครบ ผลผันแปร และคะแนนดีแต่ละเมิตข้อห้าม

◆ การรายงานผลอย่างตรวจได้

แยกการตรวจที่คำนวณด้วยโค้ดได้ออกจากการตัดสินความหมาย ระบุว่าใครประเมิน ใช้หลักฐานใด และกรณีใดยังสรุปไม่ได้ ไม่ควรนำคะแนนที่ไม่มีที่มารวมกับผลใหม่

จากจุดนี้ ผู้อ่านควรอธิบายได้ว่า Agent ใช้ข้อมูลใด ลงมือผ่านอะไร ถูกจำกัดตรงไหน และหลักฐานใดทำให้เรียกว่างานสำเร็จ ภาคผนวกต่อไปช่วยนำคำถามเหล่านี้กลับไปอ่าน Lab

อ่านเพิ่มเติม [03] OpenAI: Evaluation best practices

ภาคผนวก ก - อ่าน Lab ให้เห็นกลไก

สไลด์ PART 1 เชื่อม Lab 1-4 ตามลำดับด้านล่าง ใช้ลิงก์ของผู้สอนเป็นจุดเริ่มต้น และตรวจเวอร์ชันก่อนทำตาม หนังสือไม่ได้รับ API หรือใช้เครดิตโมเดลเพื่อสร้างเล่นนี้

Lab	สิ่งที่ควรเรียนรู้	หลักฐานที่ควรเก็บ
1 · Setup	Environment และ Dependency พร้อม	เวอร์ชัน โค้ด Directory และผลตรวจติดตั้ง
2 · LLM	Request/Response และ Usage	Input, Output, Endpoint และ Usage จริง
3 · Agent Loop	โมเดลขอ Tool แล้ว Runtime ทำงาน	Call ID, Arguments, ผล และรอบต่อไป
4 · Hooks	กฎแทรกใน Loop และผลของลำดับ	Event, Decision, Payload และทางหยุด

จุดอ่านจริงใน Lab 3

เริ่มที่ `run_agent` แล้วตาม `llm.chat` ไปยัง `msg.tool_calls` จากนั้นดูการบันทึกข้อความ Assistant การอ่าน Arguments และ `dispatch` ผลถูกผูกด้วย `tool_call_id` ก่อนวนกลับ ถ้าไม่มี Tool Call โค้ดตัวอย่างคืนข้อความ ส่วนครบจำนวนรอบจะคืน `None` [C1]

! รายละเอียดที่มีผลต่อคำตอบ

`get_time` ใช้เวลาของเครื่องที่รันโค้ด และข้อความที่คืนไม่มี Timezone กำกับ จึงต้องรู้ Timezone ของ Runtime ก่อนนำไปตีความเป็นเวลาท้องถิ่นของผู้ใช้

ฟังก์ชัน `calculate` ใน Lab จำกัดตัวอักษรก่อนใช้ `eval` เพื่อสาธิตเครื่องคิดเลข ข้อจำกัดนี้ไม่ควรถูกนำไปอ้างว่าเป็นตัวประเมินนิพจน์ที่ปลอดภัยครบทุกกรณีสำหรับระบบจริง

อ่านเพิ่มเติม [C1] Course source: Lab 3: agent_loop.py · [S1] Aekanun Thongtae: PART 1: Harness & Agent Loop (37 slides)

ภาคผนวก ข - คำถามและแนวคำตอบ

1. โมเดลขอ Tool แต่ Tool ยังไม่รัน งานเริ่มลงมือหรือยัง?

ยังไม่มีหลักฐาน Execution คำร้องขอเป็นข้อเสนอ ต้องตามไปดูว่า Runtime ตรวจแล้วเรียกฟังก์ชันจริงหรือไม่ รวมถึงผลตอบกลับและผลที่ปลายทาง

2. ทำไมต้องบันทึก Tool Call และ Tool Result คู่กัน?

เพื่อให้การตอบรอบต่อไปสัมพันธ์กับคำร้องขอที่เกิดขึ้นแล้ว และรักษาสัญญาของ API ต้องใช้รหัสอ้างอิงให้ถูก ไม่ส่งผลลอย ๆ ที่จับคู่กับการเรียกไม่ได้

3. Guardrail ควรอยู่ก่อนหรือหลัง Tool?

ขึ้นกับสิ่งที่ป้องกัน ถ้าห้ามผลกระทบจากการกระทำให้ตรวจก่อน Execution ถ้าจะตรวจผลหรือปิดบังข้อมูลตอบกลับให้ตรวจหลัง Tool แต่การตรวจหลังทำไม่สามารถย้อนผลที่เกิดขึ้นแล้วโดยอัตโนมัติ

4. คำตอบมีตัวเลขแล้ว ถือว่าคำนวณถูกหรือไม่?

ยังไม่ได้ การพบ Digit เป็นเพียงเงื่อนไขรูปแบบ ต้องตรวจค่ากับผลคำนวณและโจทย์จริง ส่วนการยืนยันว่าใช้ Tool ต้องดู Trace เพิ่ม

5. MCP Server มี Tool ใหม่ ทุก Client เห็นทันทีหรือไม่?

ไม่ควรสมมติเช่นนั้น ต้องดูการค้นพบ Refresh/Cache และความสามารถของ Client แต่ละตัว ทดสอบจากรายการ Tool ที่ระบบเห็นจริง

→ ใช้ตรวจสอบความเข้าใจ

แนวคำตอบนี้ใช้สำหรับทบทวน ไม่ใช่คะแนนของผู้เรียน ข้อที่ตอบได้ควรอธิบายต่อได้ว่า “จะดูหลักฐานตรงไหน”

อ่านเพิ่มเติม [O1] OpenAI: Function calling · [O4] OpenAI: Guardrails and human review · [O5] OpenAI: Remote MCP servers · [C3] Course source: Lab 4: agent_loop_hooks.py

ภาคผนวก ค - คำศัพท์ใช้งาน

คำ	ความหมายในหนังสือ
Harness	ระบบรอบ โมเดลที่จัดการบริบท เครื่องมือ สถานะ และขอบเขต
Agent Loop	วงจรเลือกขั้นตอน ลงมือ รับผล และตัดสินใจต่อ
Tool Call	คำร้องขอใช้ความสามารถ พร้อม Arguments
Execution	การทำงานจริงของ Runtime หรือบริการ
Observation	ข้อมูลผลที่ใช้ทำงานต่อหรือประเมินเป้าหมาย
Schema	ข้อกำหนดรูปแบบและชนิดข้อมูล
Context	ข้อมูลที่โมเดลใช้ในการเรียกครั้งนั้น
State	สถานะของงานที่ระบบเก็บและปรับปรุง
Hook	จุดที่ Runtime เปิดให้แทรก Logic
Guardrail	การตรวจหรือบังคับขอบเขตที่ระบบกำหนด
Sandbox	สภาพแวดล้อมที่จำกัดการเข้าถึงและผลกระทบของงานที่รัน
Evidence	หลักฐานที่รองรับข้ออ้างหรือสถานะงาน
Regression	พฤติกรรมที่แย่ลงหลังการเปลี่ยนแปลง
Ground Truth	ข้อมูลอ้างอิงที่กำหนดให้ใช้ตัดสินในโจทย์หนึ่ง

คำเดียวกันอาจมีขอบเขตต่างกันตาม Framework จึงควรอ่านคำจำกัดความของ API และได้จริงประกอบ โดยเฉพาะชื่อ Hook Event, Message Role และสถานะจบงาน

เอกสารอ้างอิง - OpenAI

ตรวจเอกสารวันที่ 18 กันยายน 2026 ใช้เอกสารเหล่านี้ยืนยันแนวคิดและขอบเขตที่เกี่ยวข้อง ไม่ได้อ้างว่าผู้ให้บริการรับรองหนังสือเล่มนี้

[01] OpenAI. Function calling.

<https://developers.openai.com/api/docs/guides/function-calling>

[02] OpenAI. Conversation state.

<https://developers.openai.com/api/docs/guides/conversation-state>

[03] OpenAI. Evaluation best practices.

<https://developers.openai.com/api/docs/guides/evaluation-best-practices>

[04] OpenAI. Guardrails and human review.

<https://developers.openai.com/api/docs/guides/agents/guardrails-approvals>

[05] OpenAI. Remote MCP servers.

<https://developers.openai.com/api/docs/guides/tools-connectors-mcp>

[06] OpenAI. Using tools.

<https://developers.openai.com/api/docs/guides/tools>

◆ วิธีใช้อ้างอิง

รหัสเทียมเรียบเรียงเพื่ออธิบาย ส่วน โค้ดจริงที่ยกมาระบุแหล่งและส่วนที่ตัดออก และไม่รับประกันว่า API ทุกค่ายมีชื่อ Field หรือวงจรจัดการสถานะเหมือนกัน

เอกสารอ้างอิง - Anthropic

เอกสาร Engineering อธิบายแนวคิด ส่วนเอกสาร API และ Hooks ระบุสัญญาของผลิตภัณฑ์นั้น โดยตรง ต้องแยกสองระดับนี้เมื่อพัฒนาระบบจริง

[A1] Anthropic. Building effective agents.

<https://www.anthropic.com/engineering/building-effective-agents>

[A2] Anthropic. Effective context engineering for AI agents.

<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

[A3] Anthropic. Writing effective tools for AI agents.

<https://www.anthropic.com/engineering/writing-tools-for-agents>

[A4] Anthropic. Hooks reference - Claude Code.

<https://code.claude.com/docs/en/hooks>

[A5] Anthropic. Making Claude Code more secure and autonomous with sandboxing.

<https://www.anthropic.com/engineering/claude-code-sandboxing>

[A6] Anthropic. Introducing the Model Context Protocol.

<https://www.anthropic.com/news/model-context-protocol>

[A7] Anthropic. Tool use with Claude.

<https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>

แหล่งการสอนและโค้ดประกอบ

[L1] Aekanun Thongtae. Author's Lecture Transcript, 17 September 2026.

บันทึก Plaud: “การสอน: AI Harness Engineering ตั้งแต่ LLM, Agent Loop และ Tools ถึง Hook” ใช้ข้อความถอดเสียงเป็นหลักฐานของเนื้อหาที่กล่าวในชั้นเรียน การถอดเสียงอาจคลาดเคลื่อน จึงปรับแก้และตรวจสอบให้จริงประกอบ

[S1] Aekanun Thongtae. PART 1: Harness & Agent Loop (37 slides).

<https://docs.google.com/presentation/d/1fGi6Nb1aZsbfTjKdh3tJ-nbscjOwFbyfcAHYV2HWcJw/edit>

ชุดสไลด์ที่ผู้เขียนส่งให้ตรวจอ่านมี 37 หน้า อ่านแบบไม่แก้ไข และไม่ถือว่าสิ่งที่ปรากฏในสไลด์ทุกส่วนถูกสอนจบในวันที่ 17 กันยายน

[C1] Course source. Lab 3: agent_loop.py.

https://github.com/aekanun2020/student-2026-AI-Harness-Engineering/blob/9af02acc68614fd0214cfe160c17eeb9591f832d/labs/lab3_agent_loop/agent_loop.py

[C2] Course source. HookManager and HookResult.

<https://github.com/aekanun2020/student-2026-AI-Harness-Engineering/blob/9af02acc68614fd0214cfe160c17eeb9591f832d/labs/core/hooks.py>

[C3] Course source. Lab 4: agent_loop_hooks.py.

https://github.com/aekanun2020/student-2026-AI-Harness-Engineering/blob/9af02acc68614fd0214cfe160c17eeb9591f832d/labs/lab4_hooksmiddleware/agent_loop_hooks.py

ภาพและรูปแบบอ้างอิง

AI Harness Engineering - Version 2, Aekanun Thongtae, August 2026. ไฟล์ PDF 74 หน้า que ผู้เขียนจัดให้ ใช้เป็นต้นแบบการจัดหน้า สี ภาพ และสัญลักษณ์ เนื้อหาของเล่มเดิมไม่ได้ถูกนำมาอ้างว่าเป็นคำบรรยายวันแรก

Source Code อ้าง Commit ที่ตรวจอ่านจริงในวันที่ 18 กันยายน การอ่านโค้ดยืนยันโครงสร้างที่ระบุ แต่ไม่ใช่หลักฐานว่าโค้ดชุดนั้นผ่านการทดสอบ Integration หรือพร้อมใช้ Production